

Configurable Full System Vertical Profiling and Analysis for Android-based SoC

Prof. Tien-Fu Chen

蘇資翔、周成德、鄭元鈞、吳瑋珊、蔡孟庭

[†]Department of CS
National Chiao Tung University
Taiwan



What is inside of your mobile phone?

Expanding Areas for SoC Integration

2MB L2 (8064 only) aSMP Custom Krait
Web Tech Innovations **CPU**
High Performance Floating Point Custom System Interconnect

FlexRender™ Technology GPGPU Compute for Imaging/Video (Renderscript, OpenCL, LLVM)
Unified Shader Architecture Stereoscopic Rendering
Low Power Innovations Accelerated WebGL Profiling Tools **GPU**

Application Processors

21 MPix Zero Shutter Lag
20x Digital zoom Ultrasound 7.1 Surround Playback
Stereo 3D video **DSP/Multimedia**
1080P HD video Computational Camera 5.1 Surround Camcorder
Browser with HTML5 and 1080p Flash Noise cancellation



Integration

LTE World Mode LTE Broadcast
LTE TDD Coexistence with connectivity
Modems VoLTE / SRVCC
RF Multi-band TD-SCDMA Advanced Receivers
Power Optimization

BaseBand

GPS/GNSS Indoor Location BT
Connectivity Miracast
.11ac Coexistence with WWAN Multiband
FM

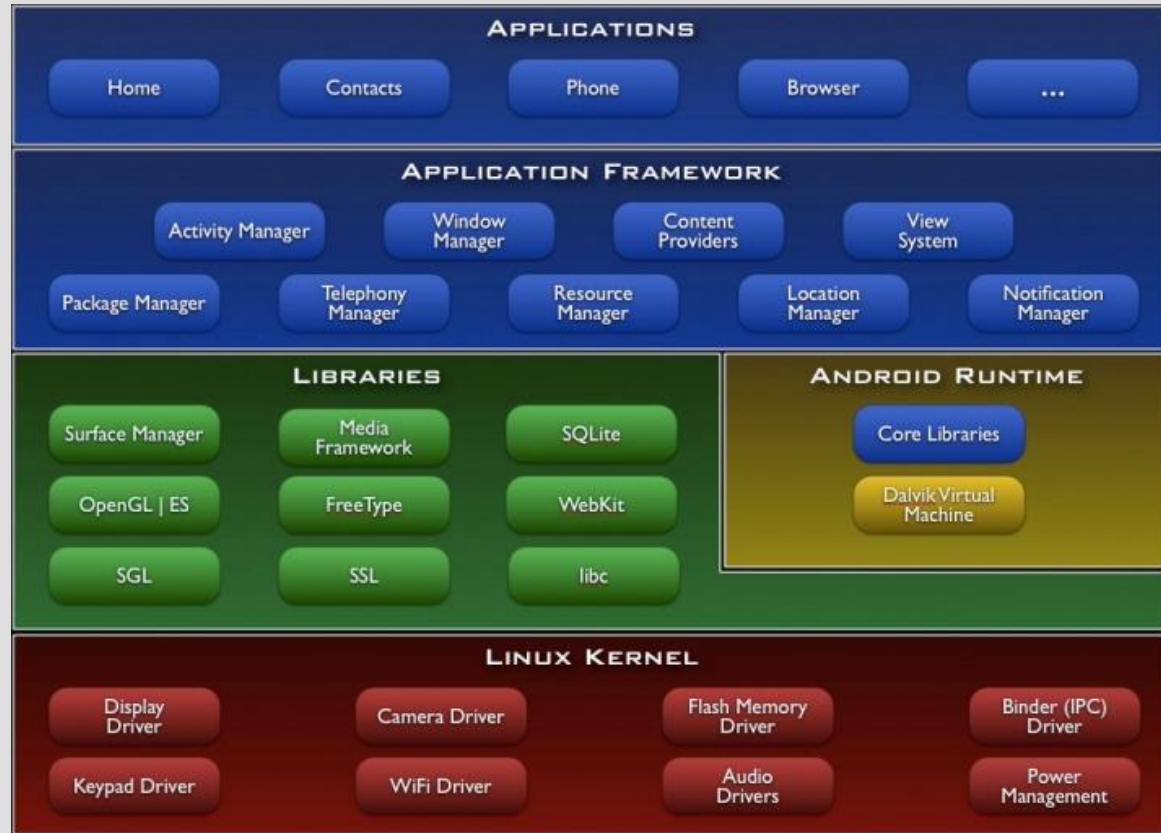
Accelerometer Proximity Ambient light
Sensors Pressure Gestures
IR Sensing Humidity
Temperature Gyroscope Magnetometer

User interface

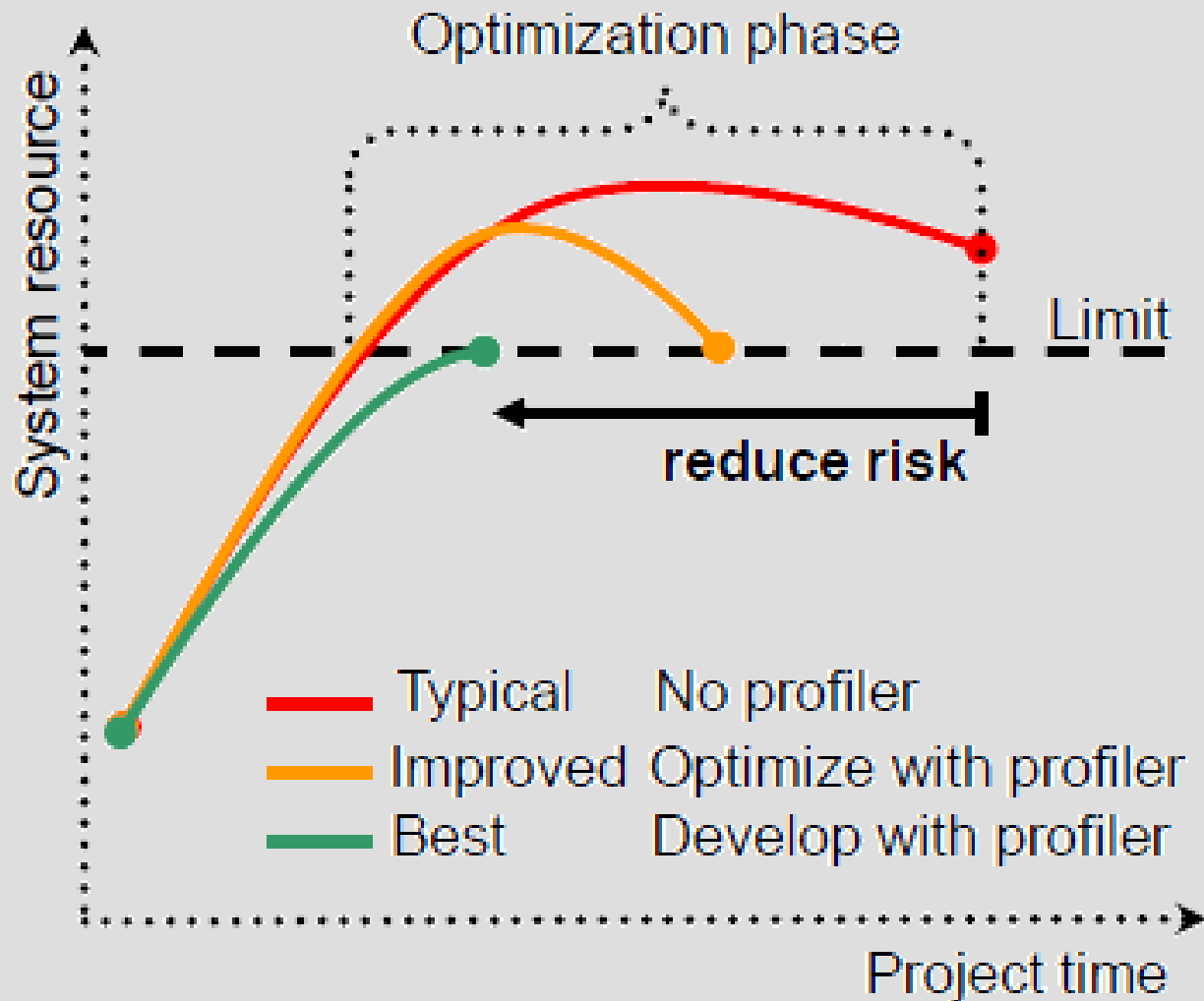
3D Content Adaptive Backlight MTPS
Displays Color correction
Wireless Display Frame Buffer Compression



What is Android inside?



Profiling is a part of design phase



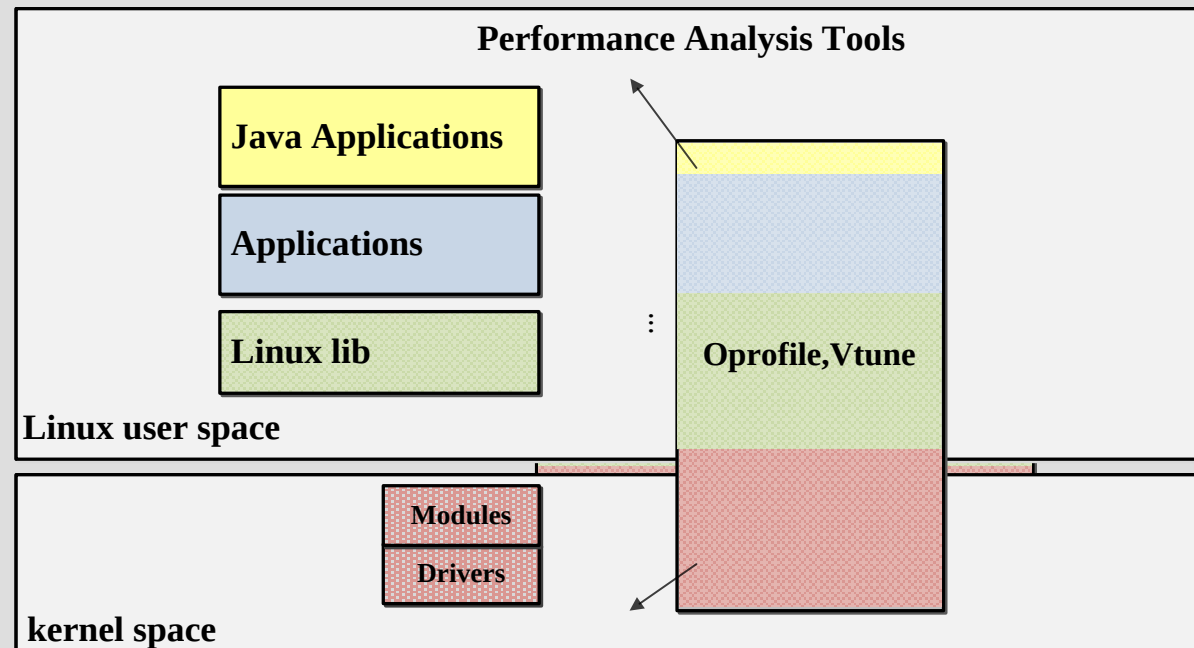
Performance profiling tools in software stacks

■ Instrumentation based profiling tools

- Profiling in each layer
- High precision
- Information barrier

■ Sampling based profiling tools

- Lower overhead
- Less precision
- System-wide
 - Symbol correlation
- Information barrier



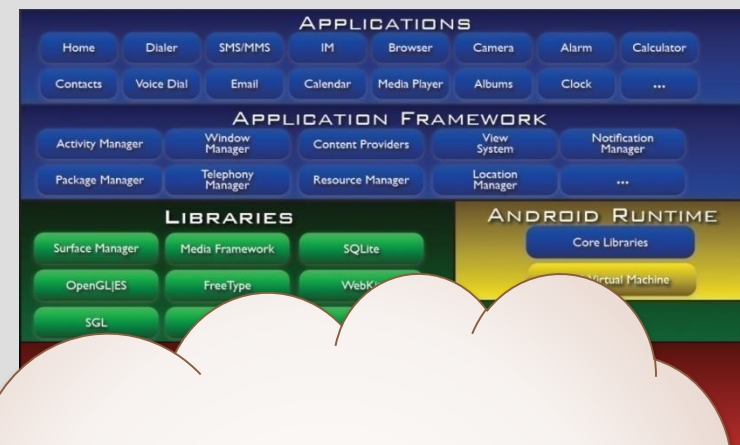
Why Vertical Profiling

1. Vertical Profiling: Integrated profiling cross software layers

- **Virtual machine, APs**
- **Libraries**
- **Kernel, driver**

2. Why vertical profiling?

- Complete view of performance
- Solves information barrier
- Combine information from different layers to understand system performance
- Correlation of performance and



**No Vertical
Profiling available
on the Android
platform**

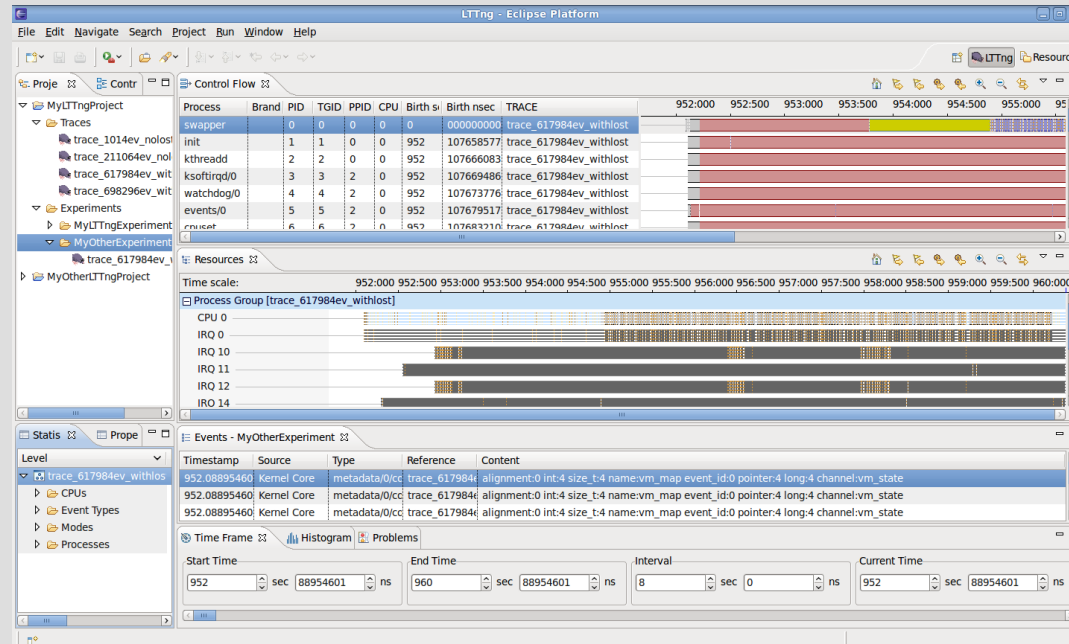
Current Android Profiling Tools



Visible Layer	Dalvik and application	Native libraries	Kernel and drivers
Traceview	Application and DVM logs	Profiling barrier	
DDSM	DVM profiling and trace		
Logcat	Application logs through activity manager		Limited to native functions or libraries calls
LTTng		User space tracing tool	Kernel space tracing tool
Ltrace		Trace for library calls	Traces system calls
Strace			System call tracing tool
Oprofile		Profiling with PMU	Profiling with PMU
This Work	bridge Unified Vertical Profiling with PMU		

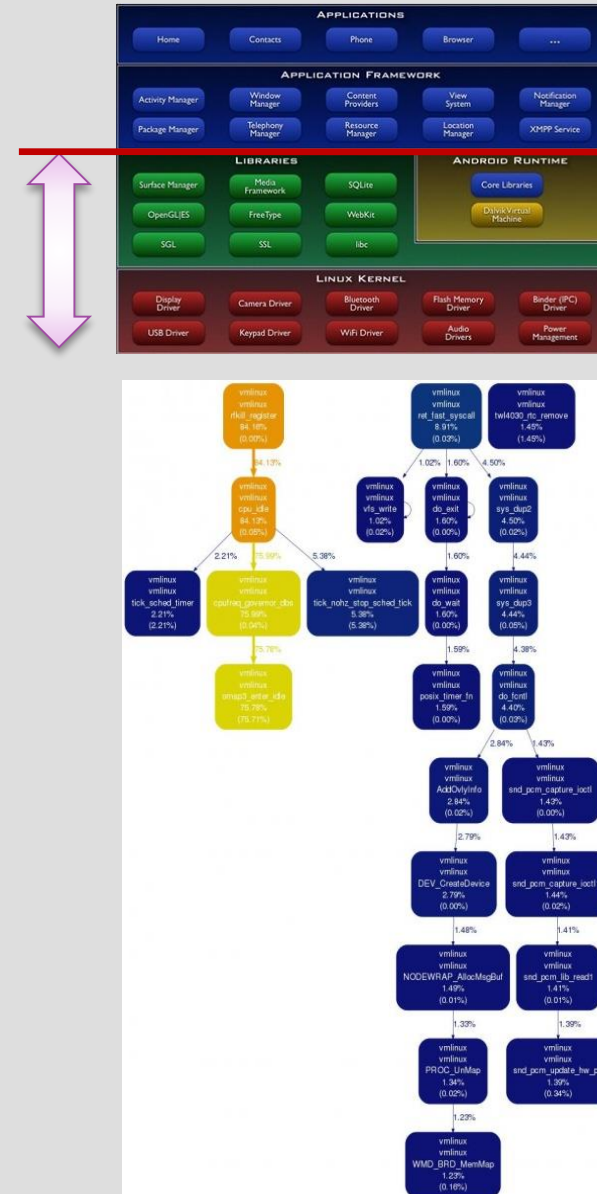
LTTng

- Not provided by Android official source code
- Only provides Linux information
- Linux user/kernel space trace tool
- Uses hooks for dynamic instrumentation
- Large trace files
- Requires source code modification

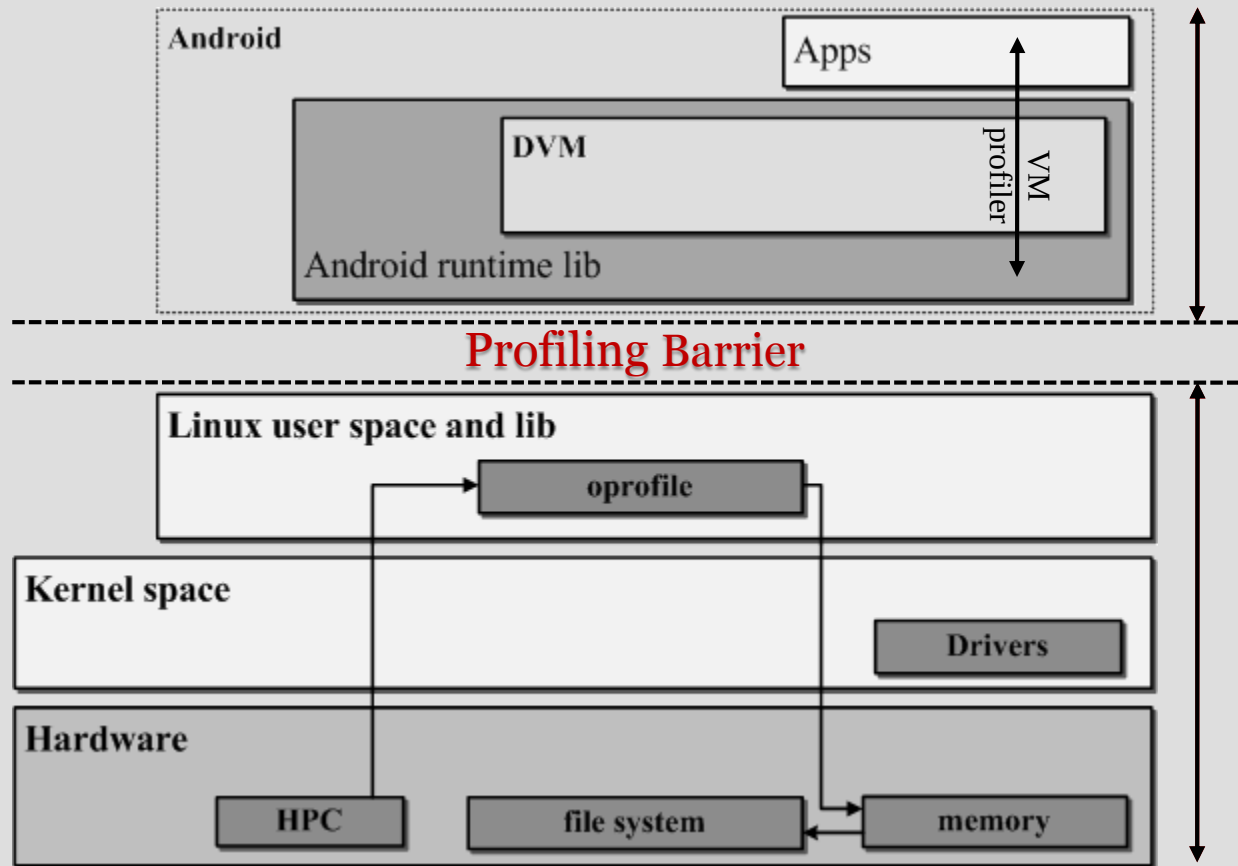


Oprofile

- Provided by Android source code
- Only provides Linux user/kernel space information
- Samples a Linux program using CPU's Performance Monitoring Unit
- No timing information
- Provides Calling information

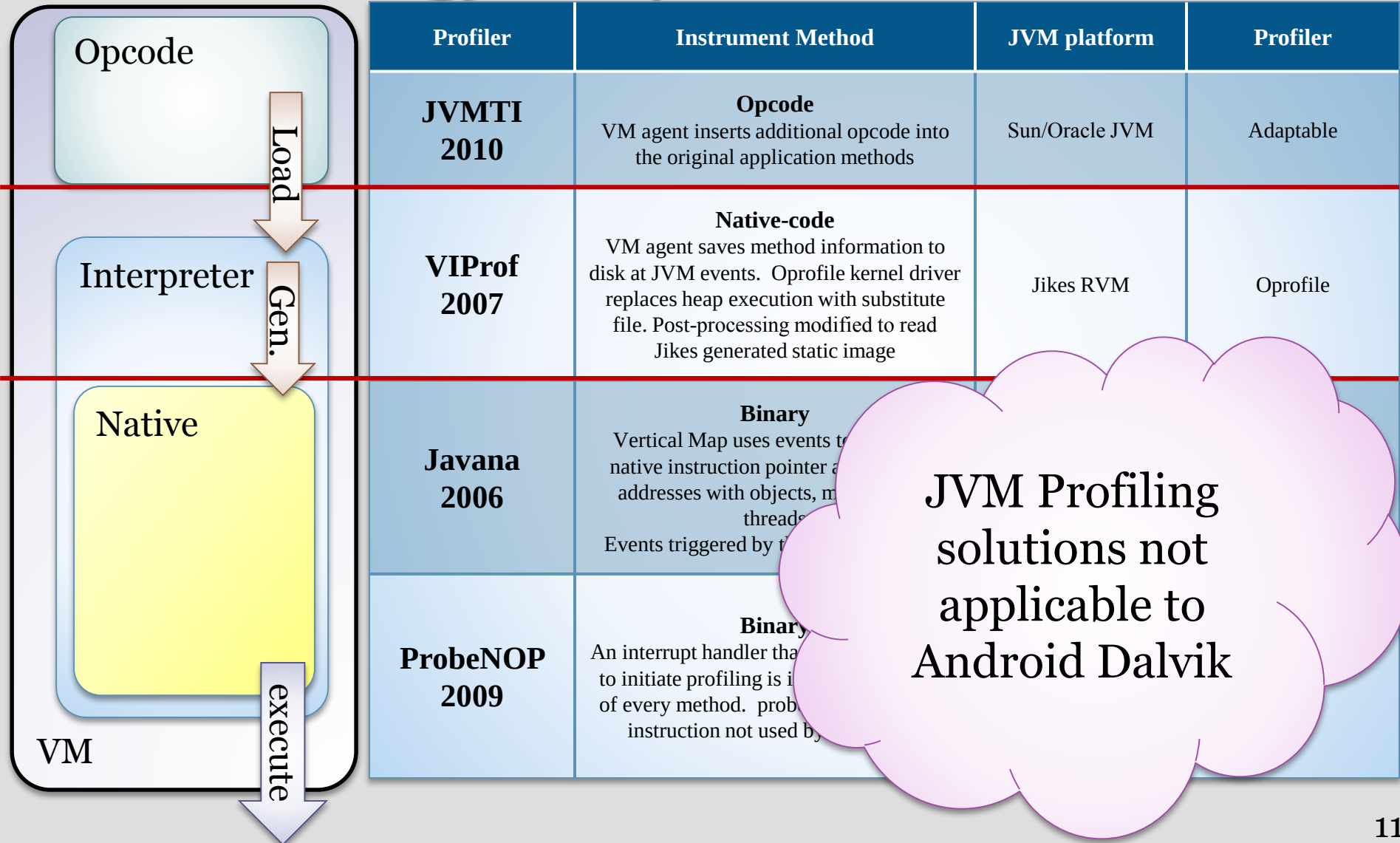


Profiling Barrier between DVM and Linux



- VM profiler can only profile VM software stack and application.
- Information barrier between DVM and kernel
- Linux Profiler can only correlate Linux user and kernel space symbol info.

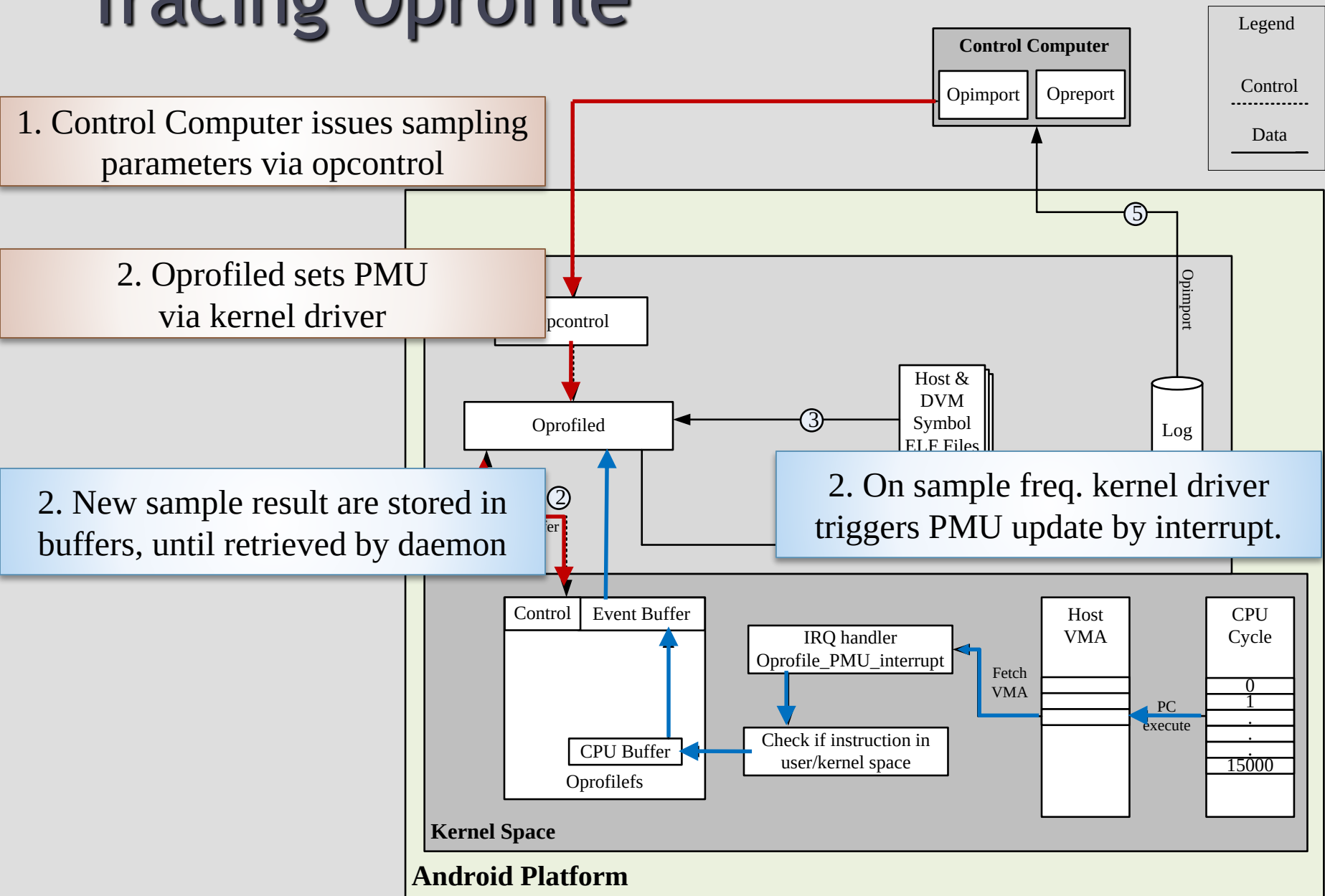
Vertical profiling instrumentation methodology comparison



Challenges of Vertical Profiling on Android

- 1. No direct runtime symbol visible to kernel**
 - No address info provided in dalvik-jit-code-cache of process
 - Expensive renaming on context
- 2. No Virtual Machine Tools Interface (JVM TI) support in Android**
- 3. Address mapping is lost because of garbage collection**
- 4. Significant overhead on each method invocation**

Tracing Oprofile



Tracing Oprofile

5. After profiling, control computer pulls the log for post process analysis

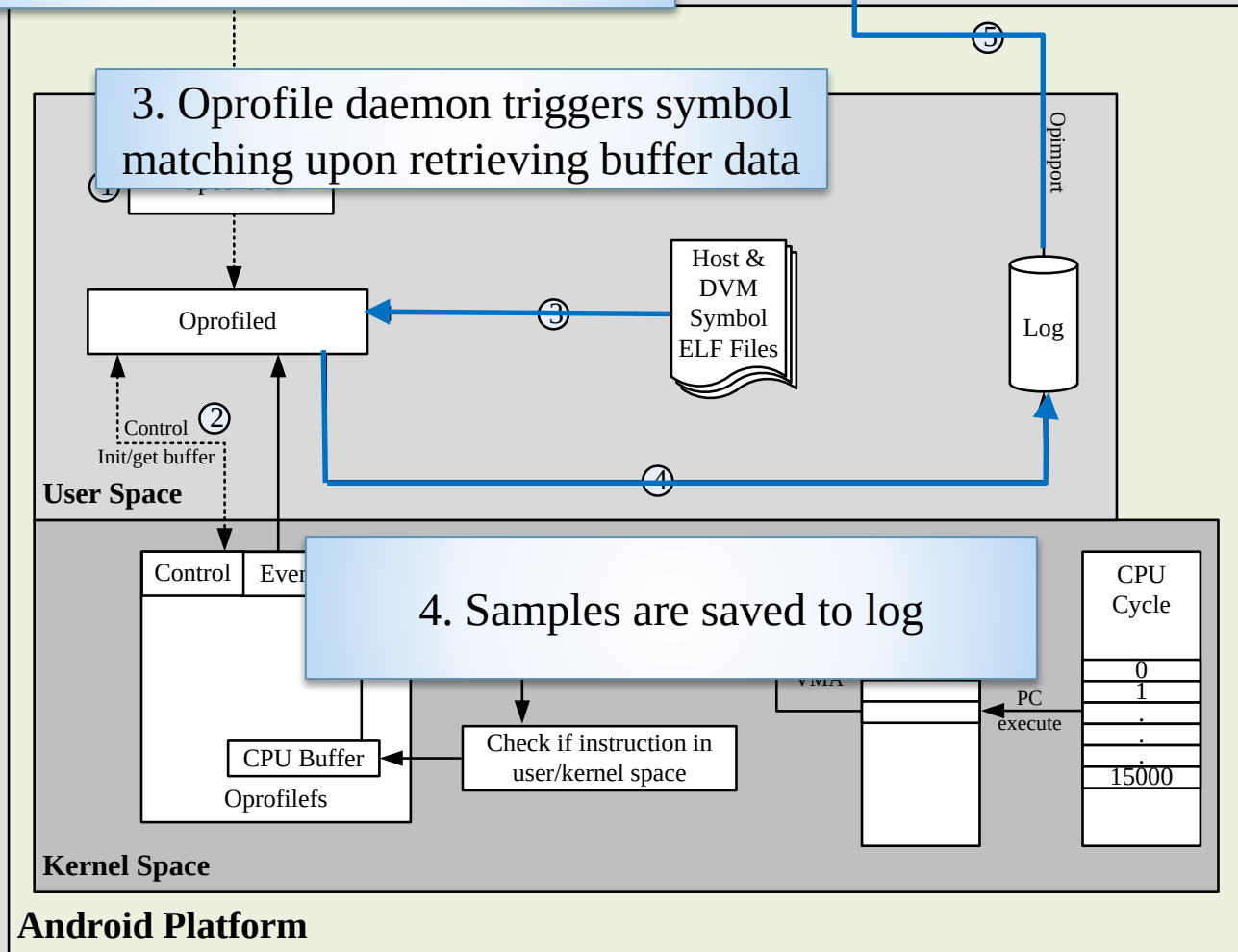
3. Oprofile daemon triggers symbol matching upon retrieving buffer data

4. Samples are saved to log

Legend

Control

Data



Our techniques solve Android Vertical Profiling Issues

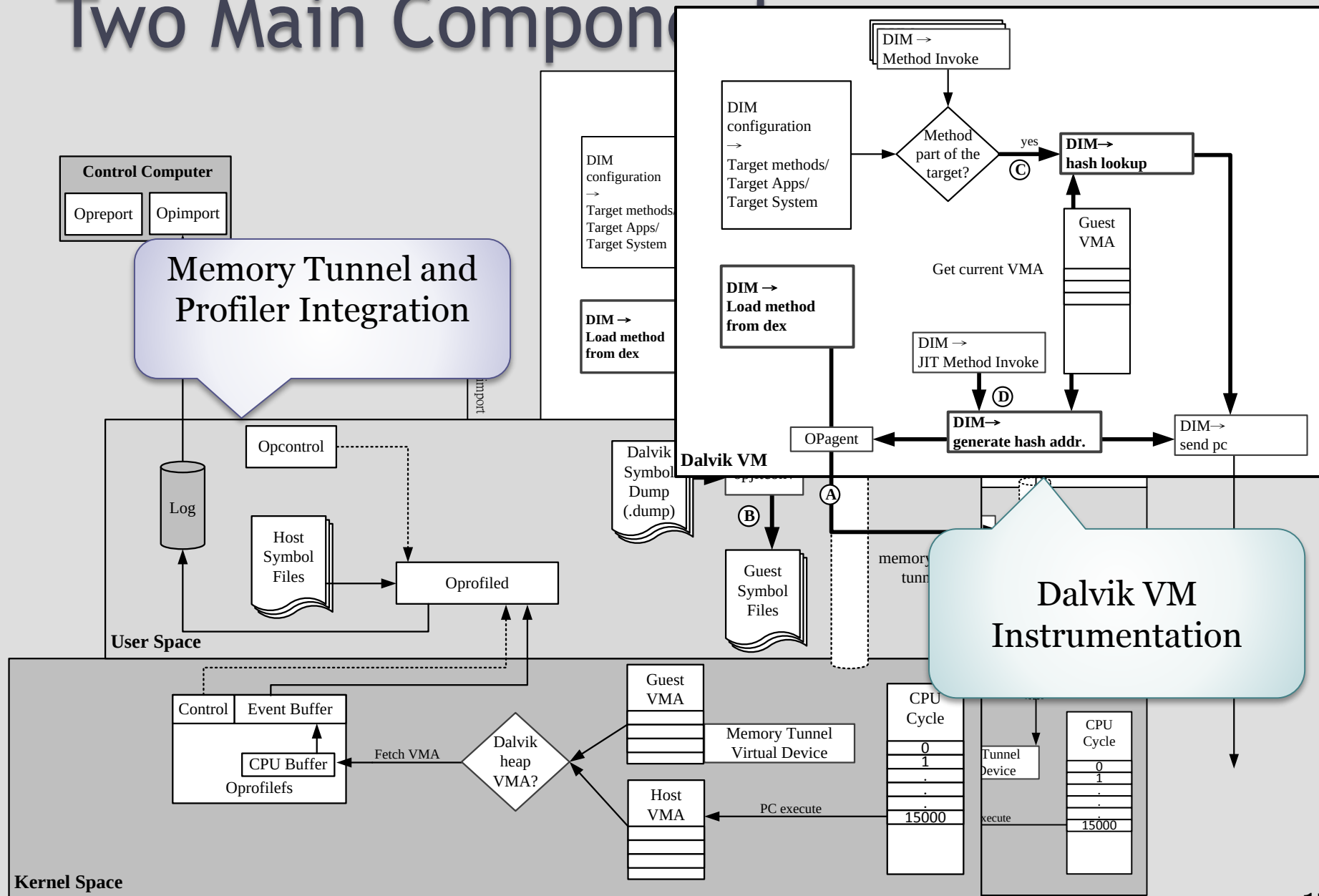
Issues	DVM	Native Program & Lib.	Kernel & Driver
Profiling barrier	Support by a Dalvik Instrum. Module		•Memory Tunnel •Oprofile kernel driver modification
Runtime symbol matching		Post-process dump file to symbol file conversion	Virtual Address Remapping
No JVMTI support			
VM profiling victims			
Lower Instr. Overhead	•Method dump at app init. •Reconfigurability	•Post-process dump file to symbol file conversion	•Memory Tunnel

Our Contributions:

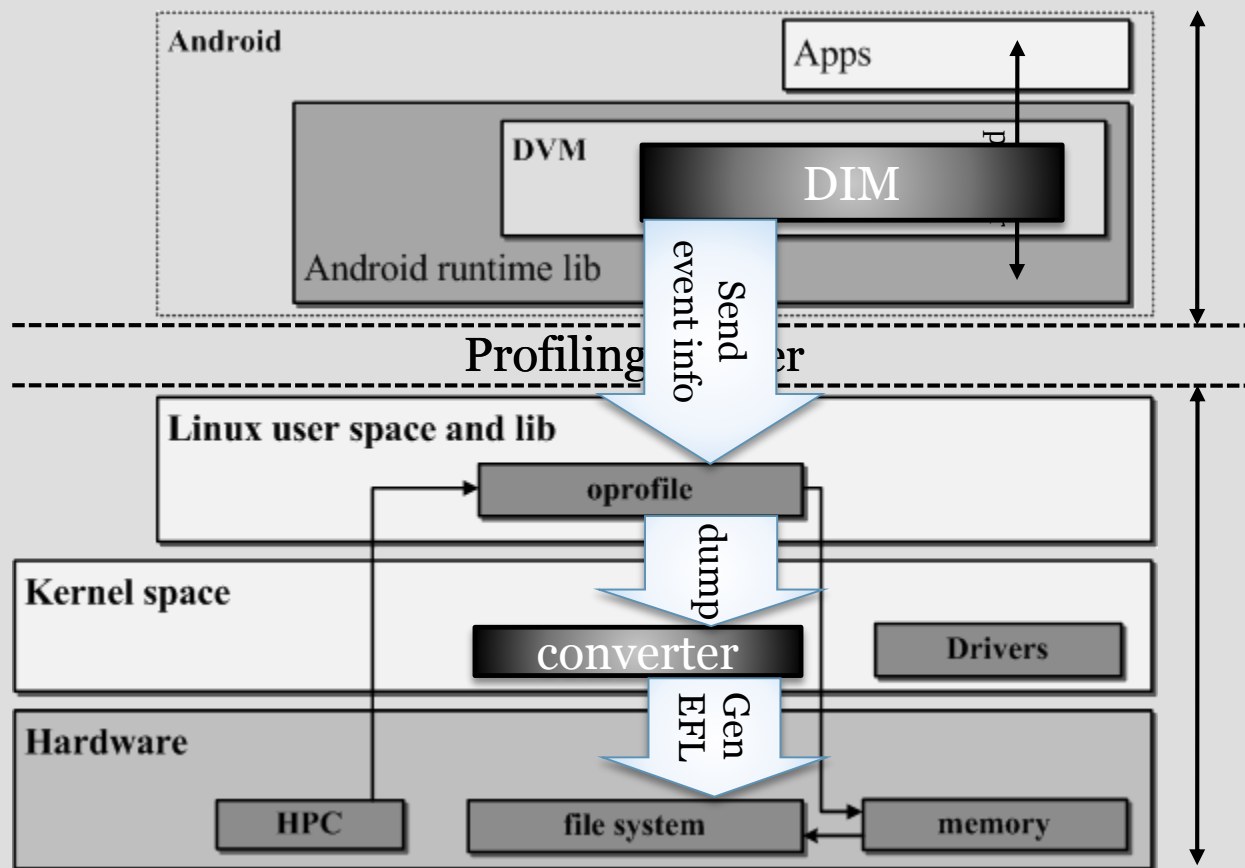
Extend Android DVM and Kernel Functionalities

1. **Develop Dalvik Instrumentation Module (DIM)**
 - **Working in Interpreter / JNI / JIT**
2. **Lazy virtual address remapping to reduce lookup time for JIT methods**
3. **Capture various profiling at either AP, process, or method levels without AP source**
4. **Configurable profiling to only targets of interests with backtrace capability**
5. **Leverage existing analysis tools: Oprofile or Perf**
6. **Build Qemu-driven SoC simulation to enable profiling-guided SoC architectural exploration**

Two Main Components

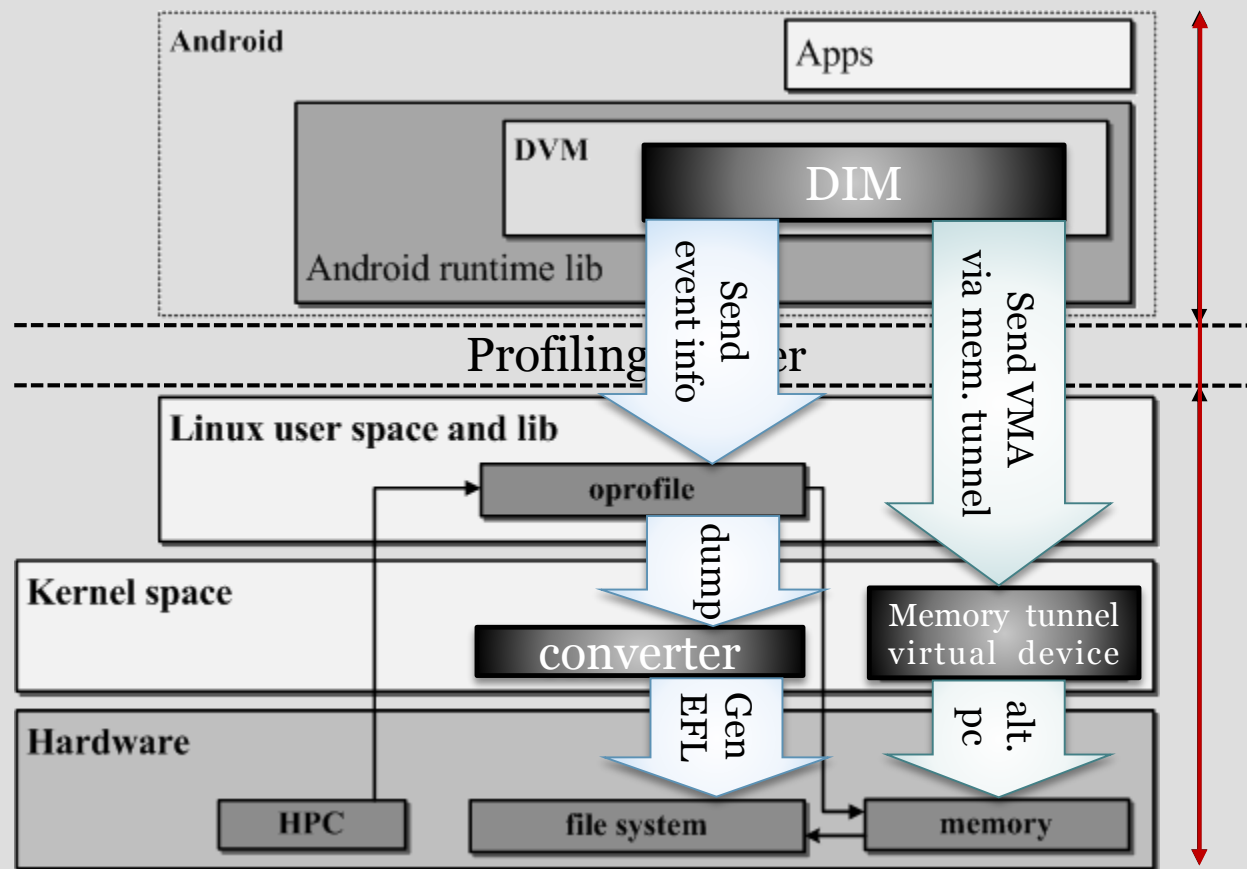


Information Crossing Profiling Barrier



- DVM Instrumentation Module (DIM).
- Write trace to file the first time a method is invoked

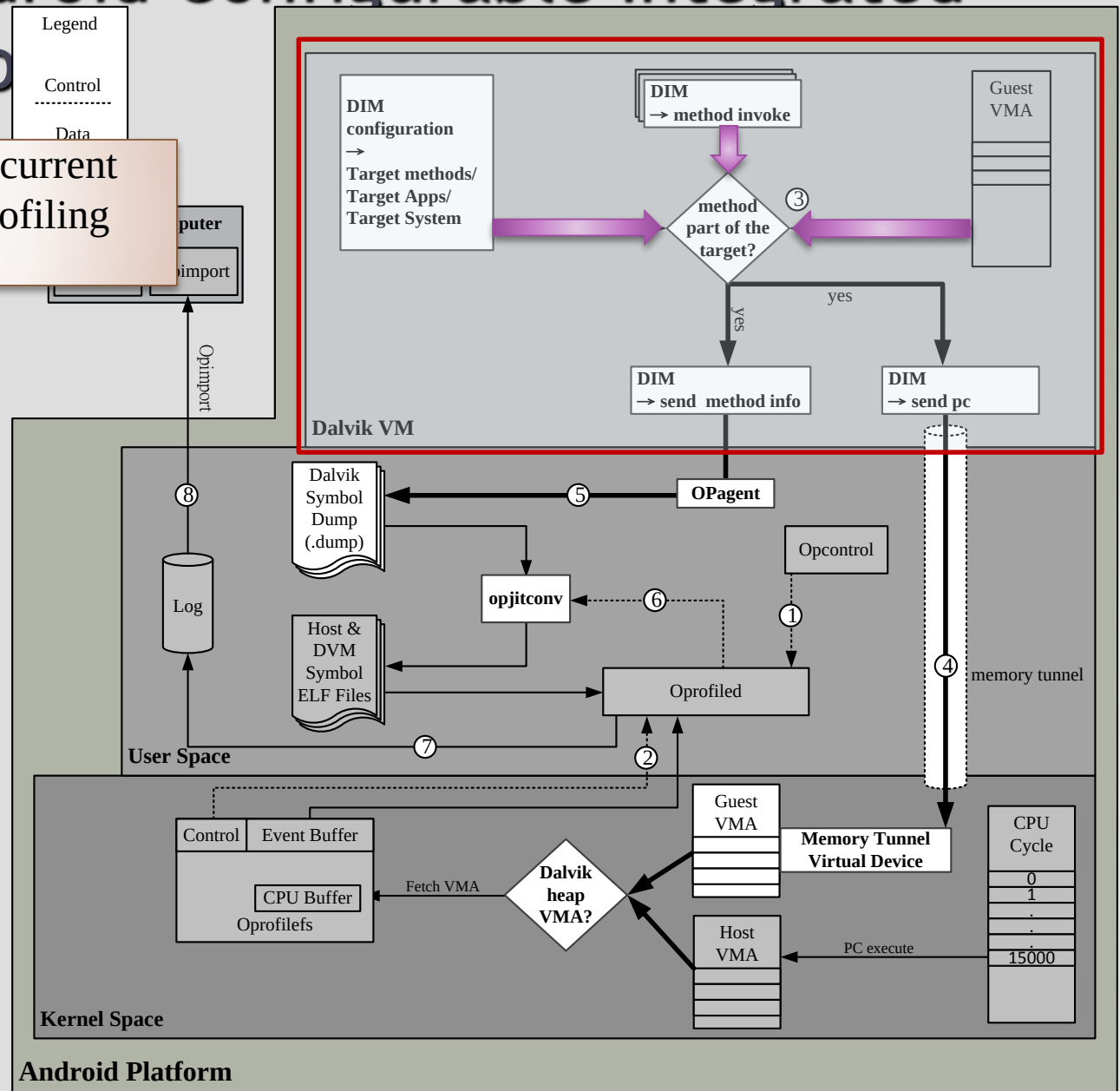
Information Crossing Profiling Barrier



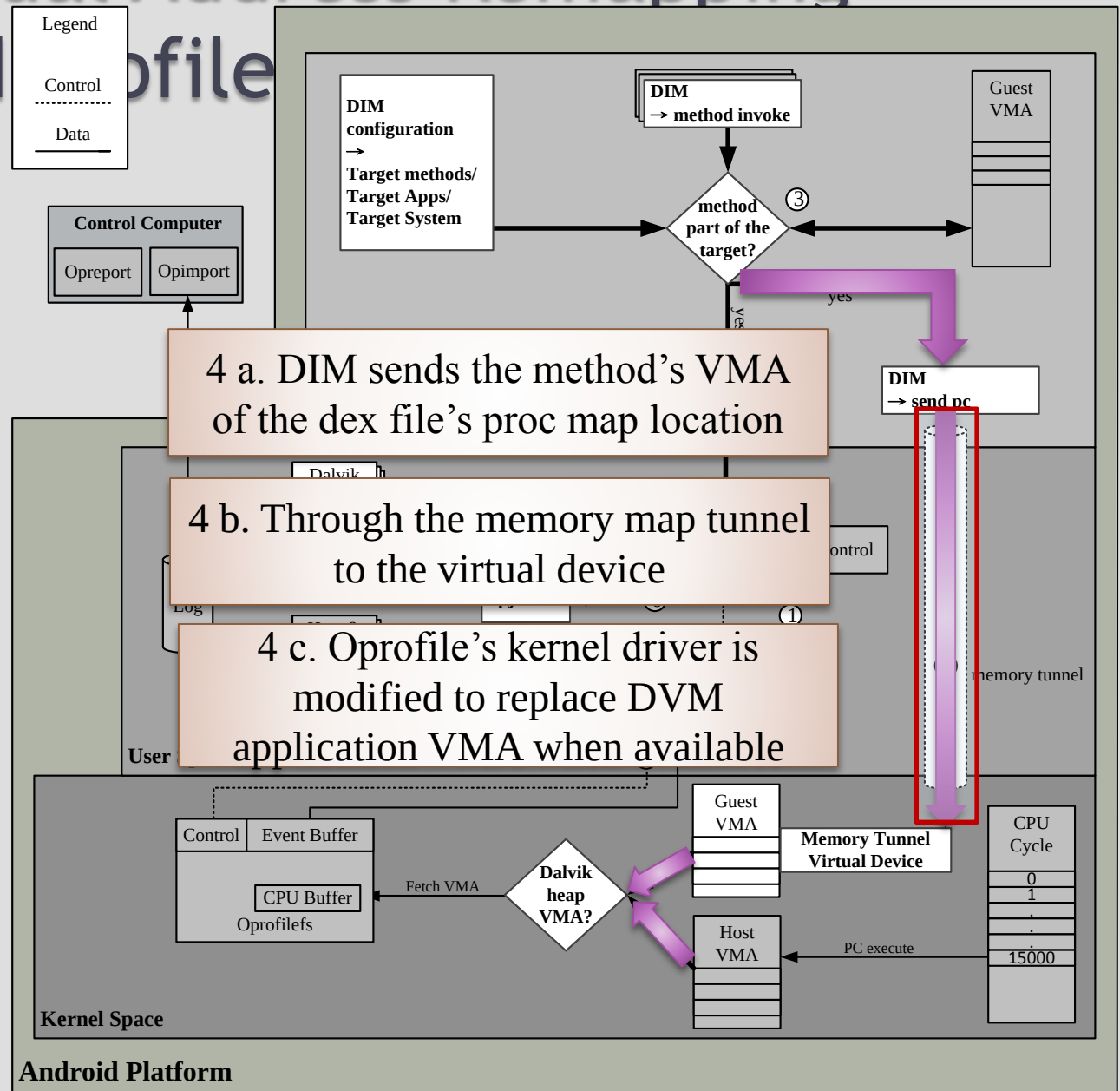
- DVM Instrumentation Module (DIM).
- Write trace to file the first time a method is invoked
- Using memory tunnel to bridge the barrier.
- Replace sample VMA.
- Complete visible symbols for Java applications, library, and kernel

Vertical Android Configurable Integrated Profiling Flow

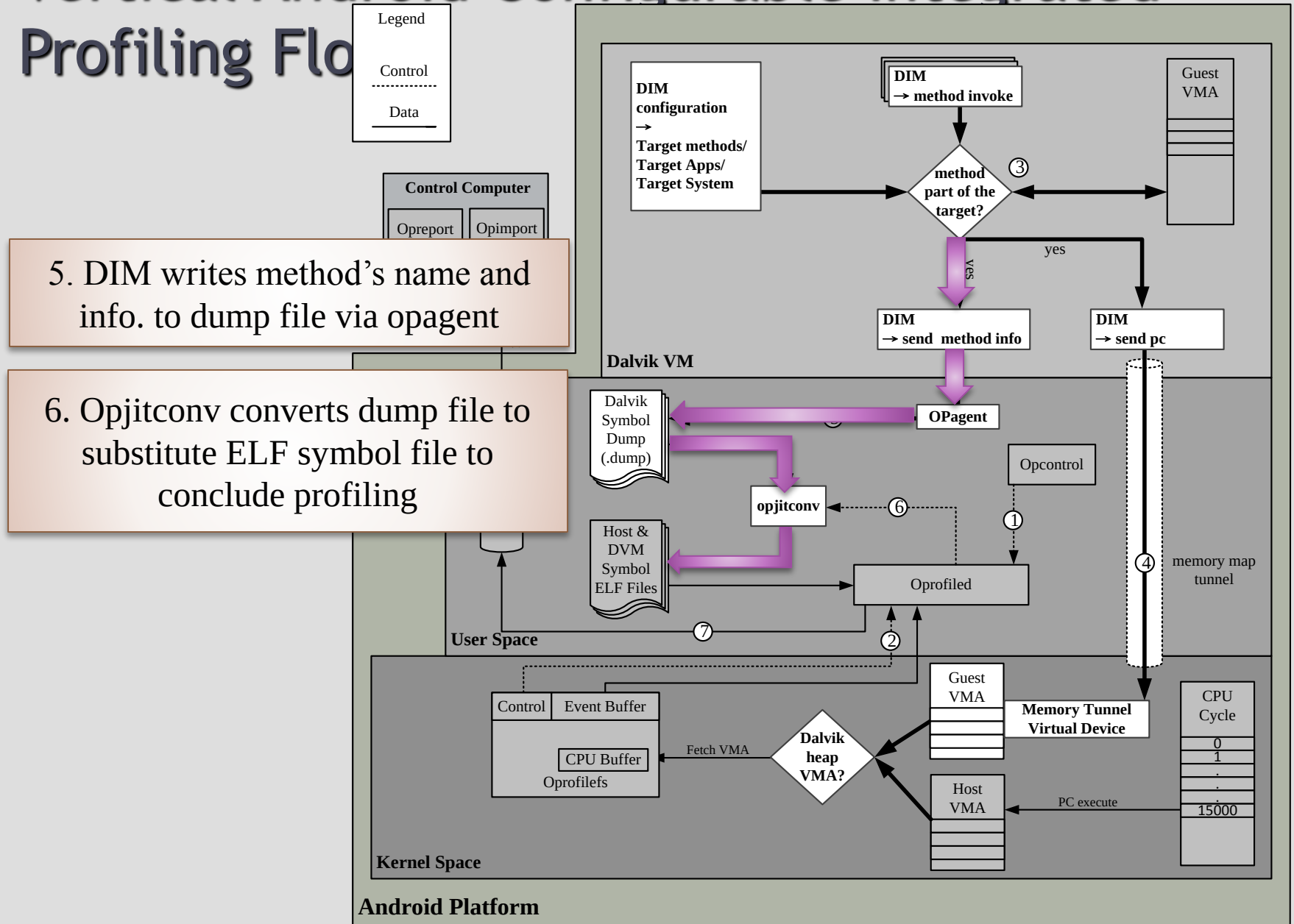
3. DIM determines if the current invoked method is the profiling target



VARI - Virtual Address Remapping Integrated Profile

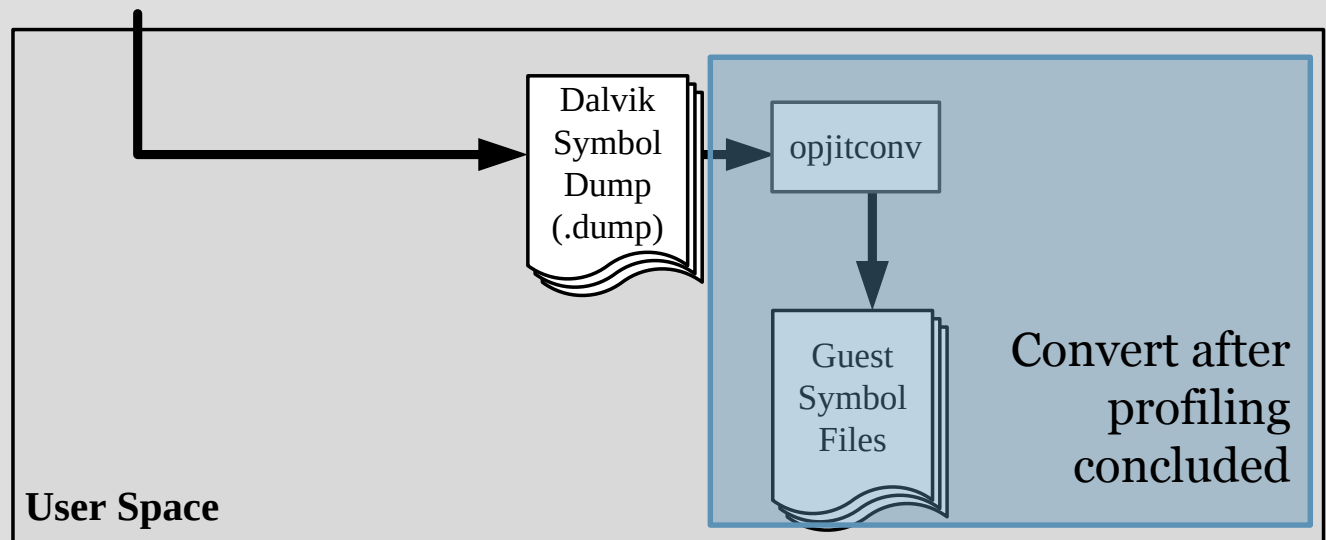


Vertical Android Configurable Integrated Profiling Flow



Post-processing ELF Symbol Conversion

- Symbol matching is performed again with oprofile's post processing tools
- Converting dump files into ELF files during runtime introduces unnecessary overhead

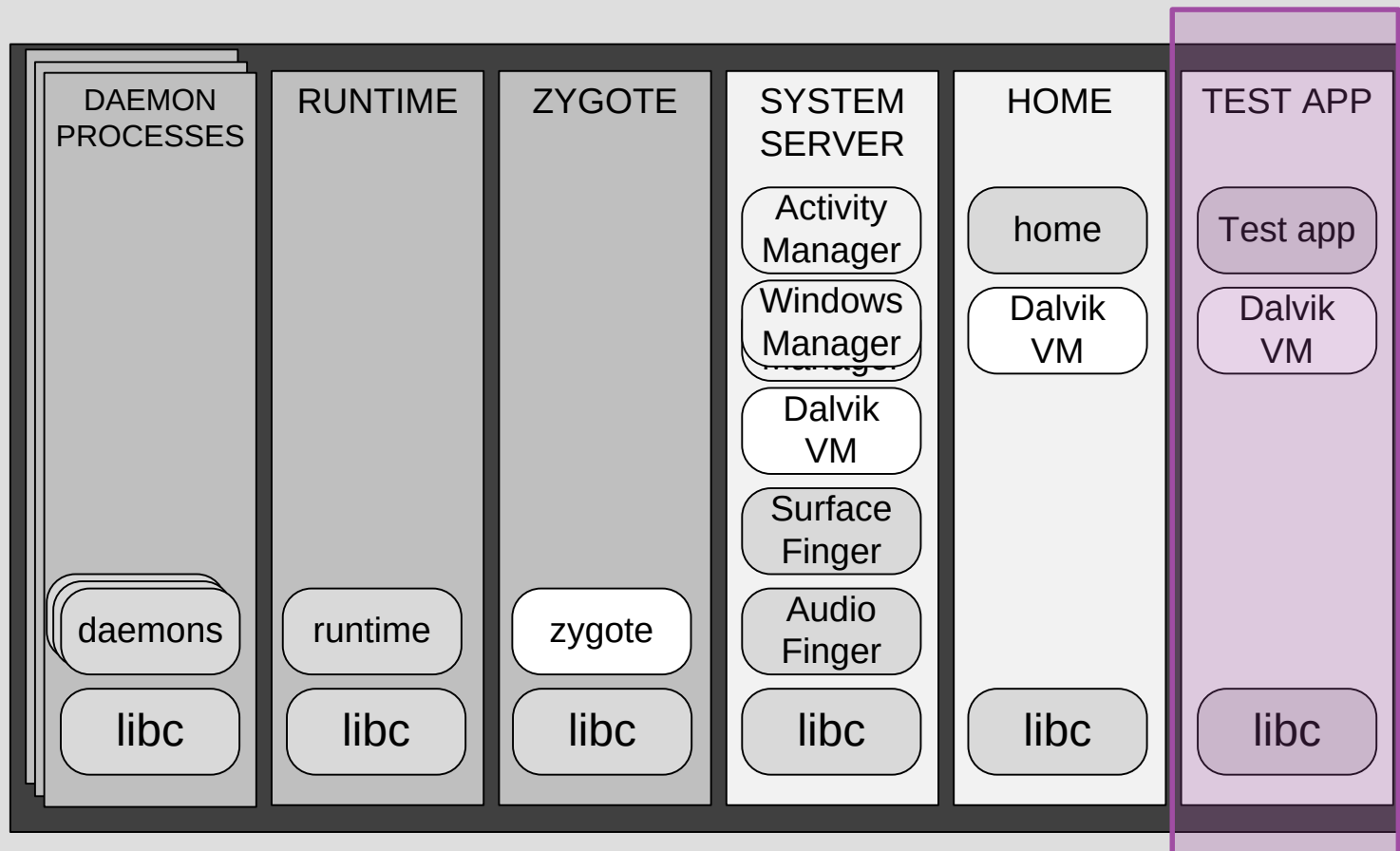


Profiling events of interest by specifying configuration

- Three level triggering available by user configuration
 - Application Triggered
 - Method Triggered
 - Service Triggered
 - Requires restarting Android
- Configure with configuration file/UI

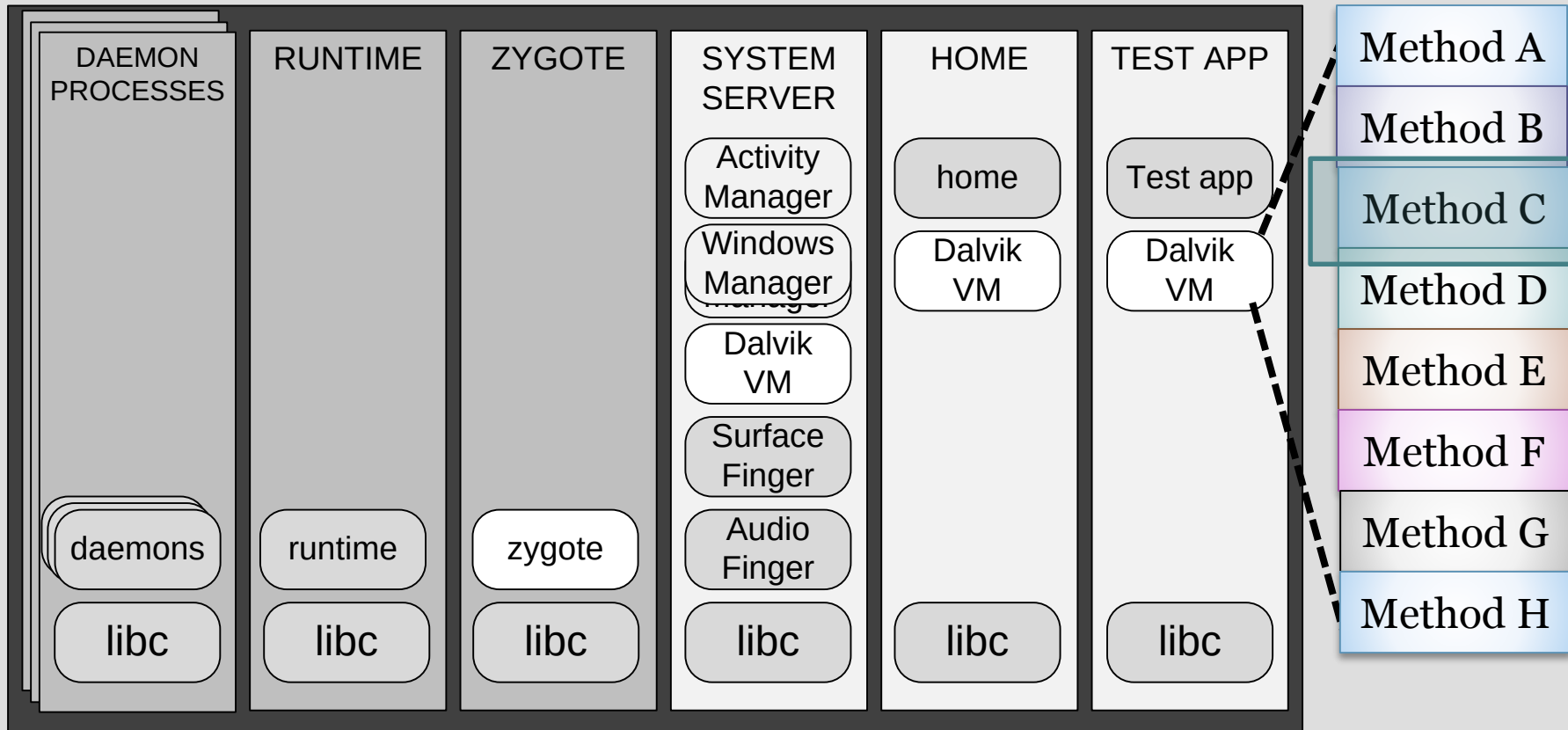
Application Triggered

- First use this granularity to identify which method is the bottleneck



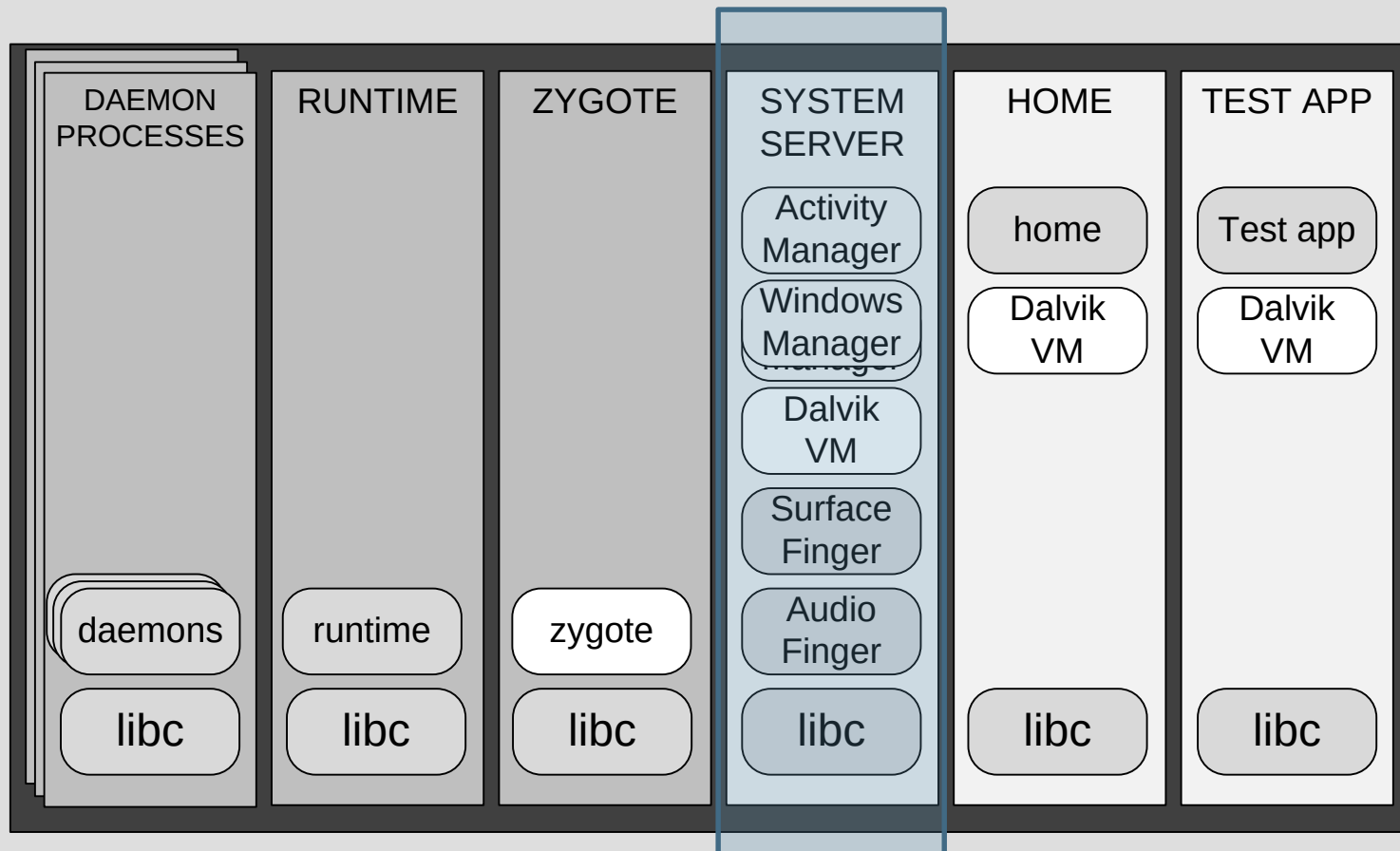
Method Triggered

- Avoids I/O interference from instrumenting other unwanted methods.

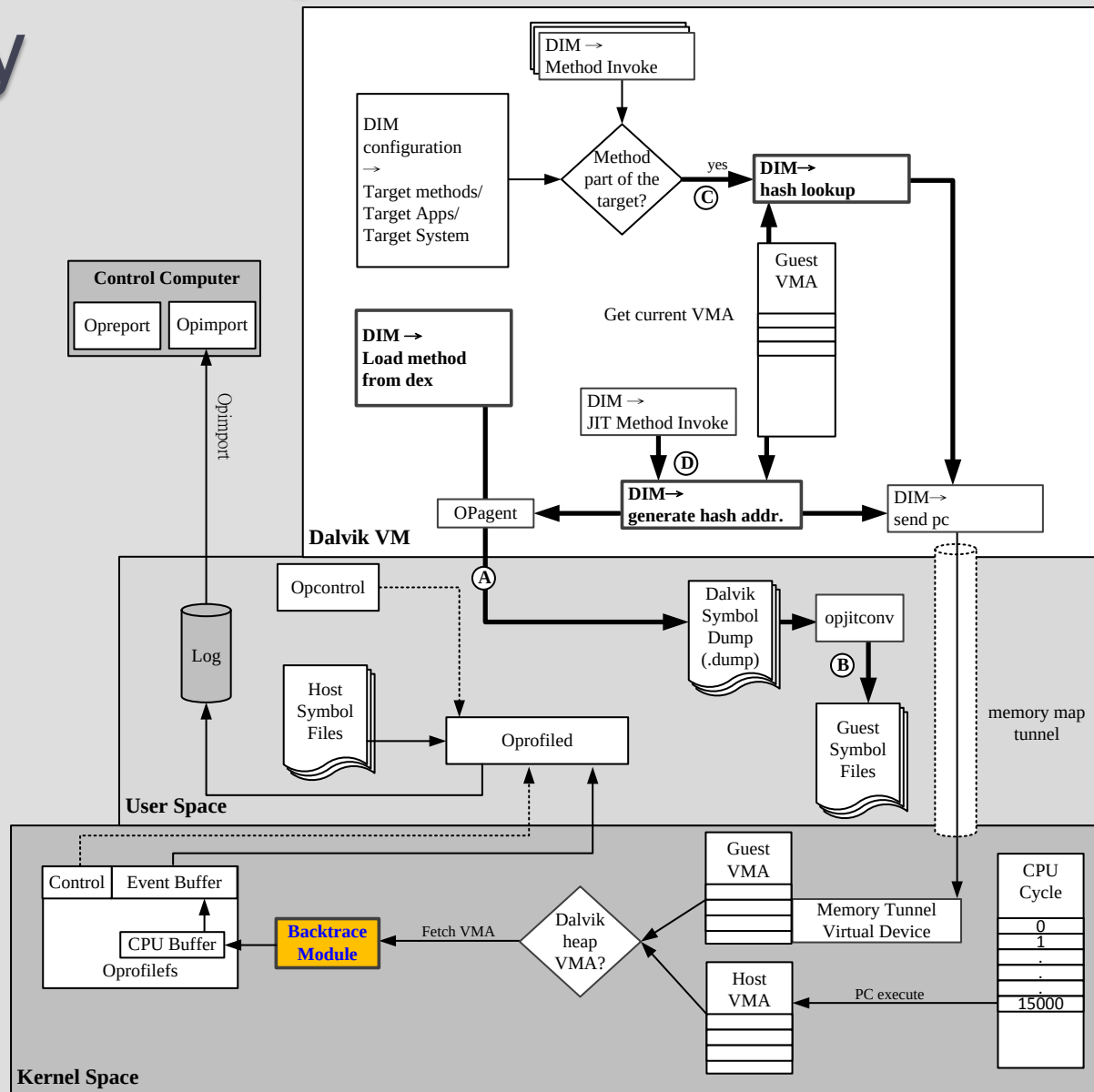


Service Triggered

- Services specific profiling can identify power hotspots and point to possible causes

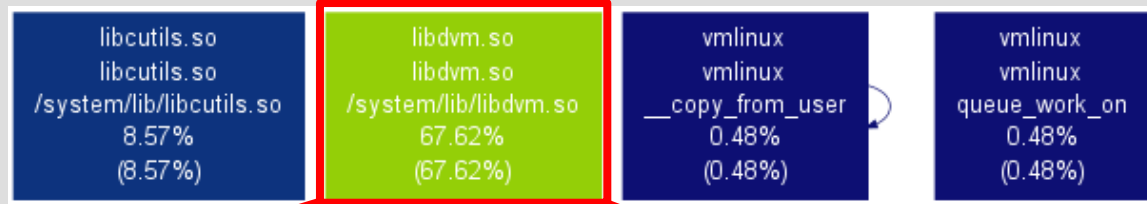


In-depth Profiling with backtrace capability

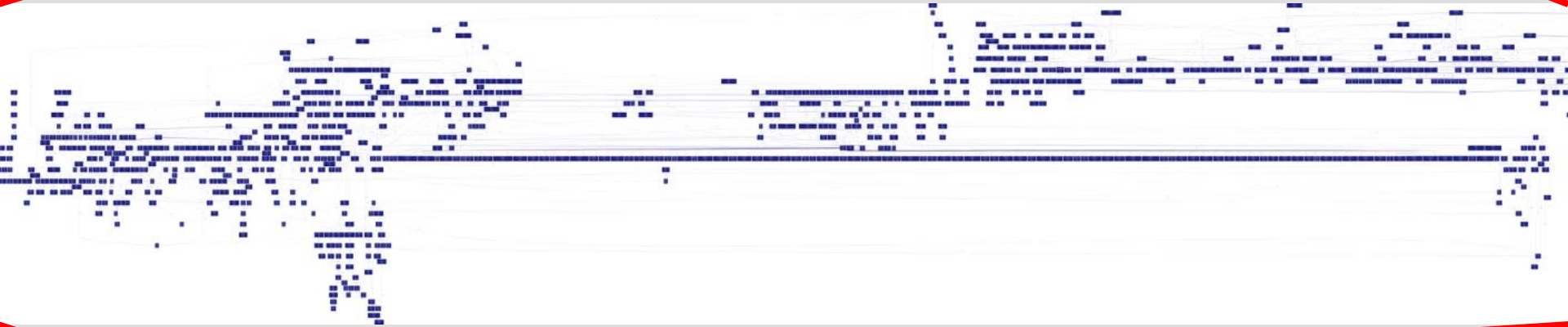


In-depth Profiling with Backtrace Capability(cont.)

•Without Backtrace



•Program View



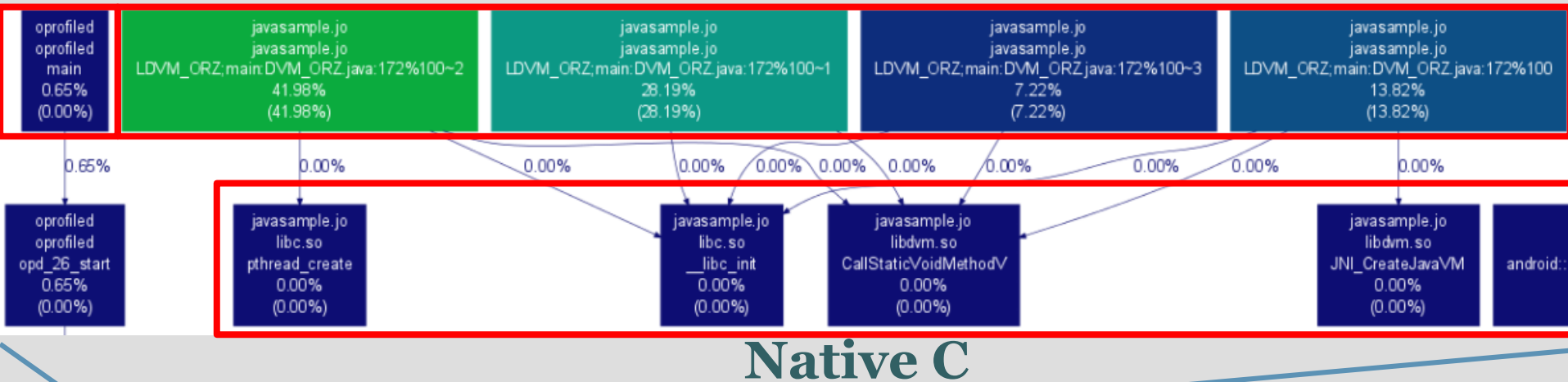
•System View



In-depth Profiling with Backtrace Capability(cont.)

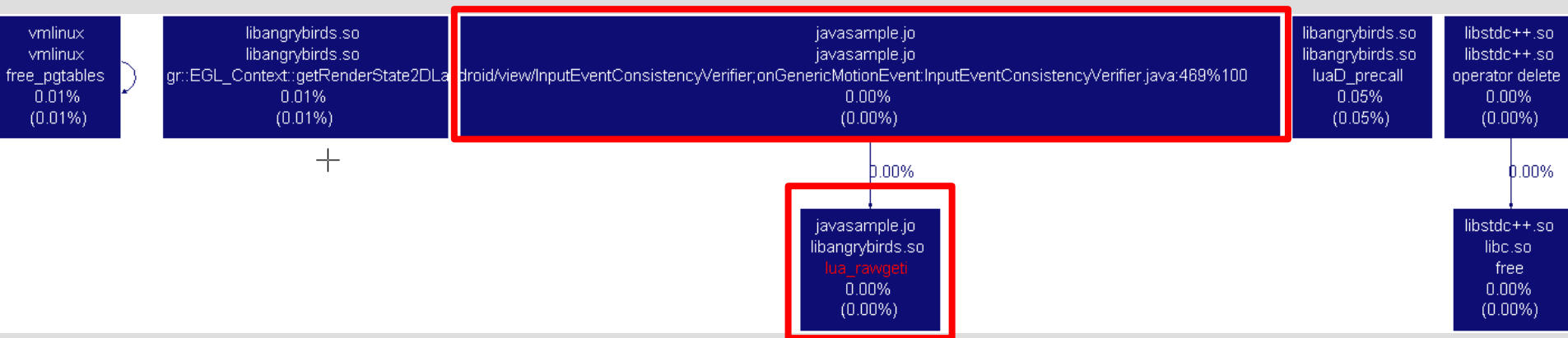
Low Overhead

Dalvik Java



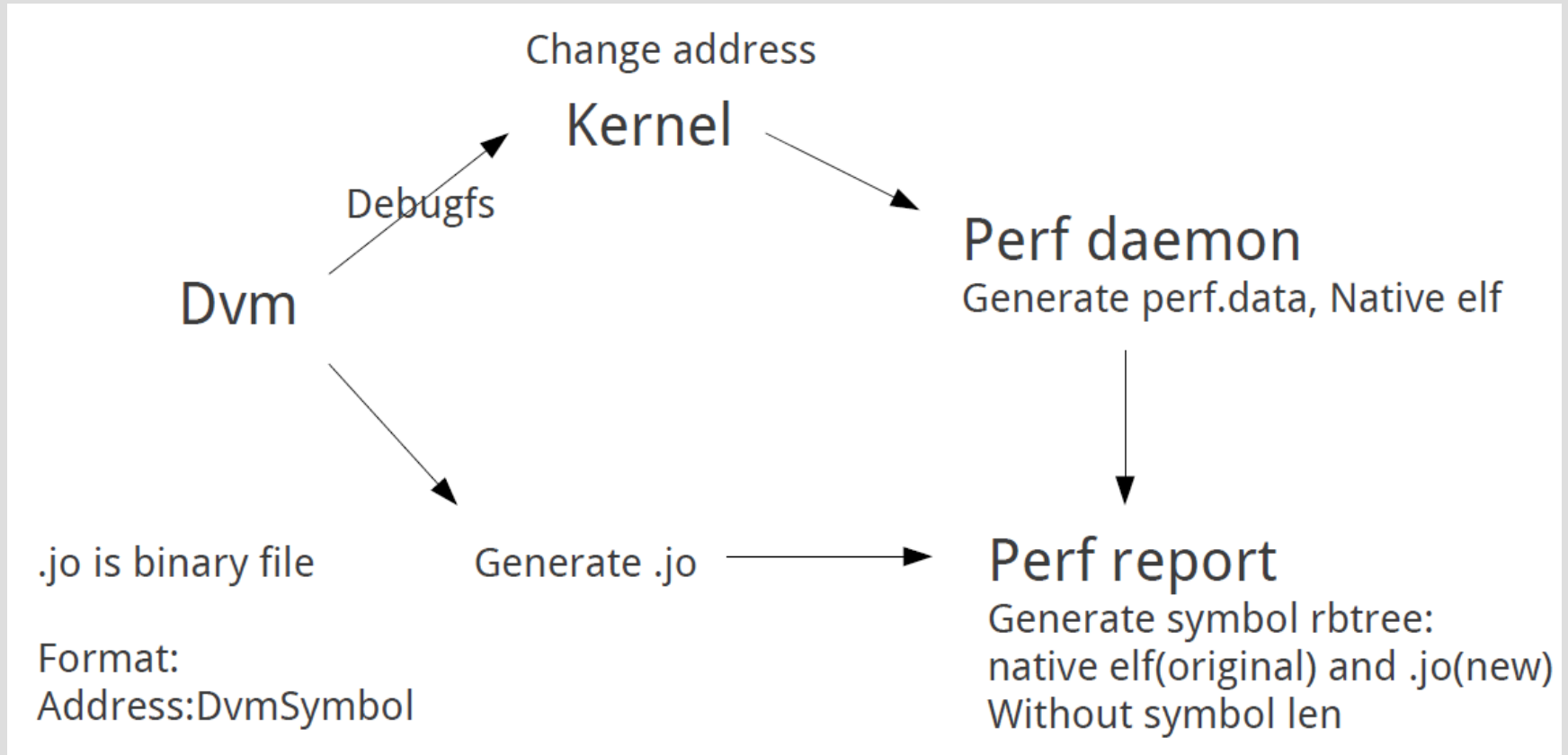
In-depth Profiling with backtrace capability(cont.)

Dalvik Java



Third Party APP JNI Library

Linking with Existing Perf Profiling Tool

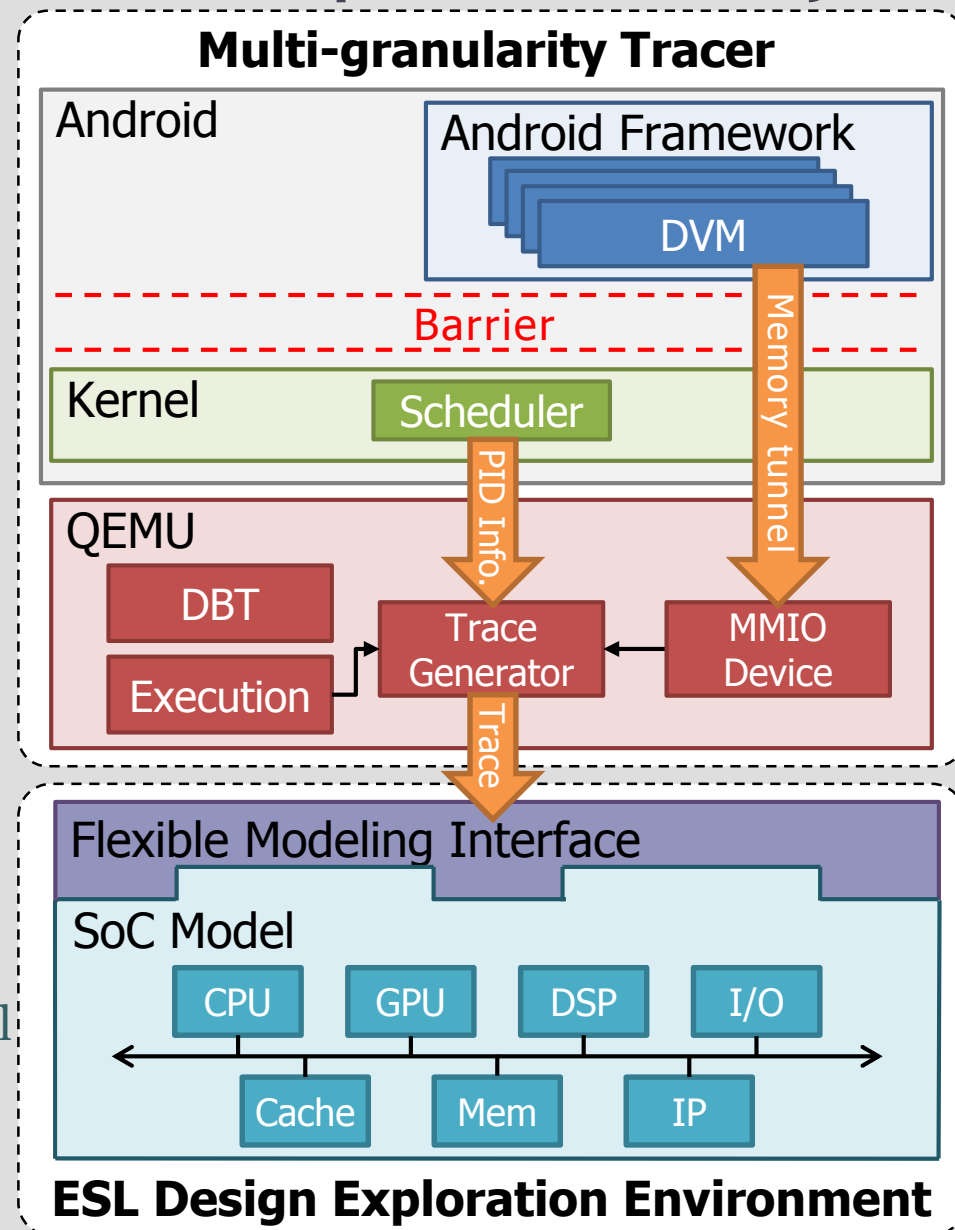


Perf vs. Oprofile for VARI

	Perf	Oprofile
Process Data	Use Red-Black tree	Use dcookie
Merge Java Info	1. Maintain separate tree at runtime 2. Must modify perf report	1. Post-processing, 2. Use original oprofile report
Porting extra lib	No	Use opagent and opjitconv to convert binary file
Dynamic instrument	Yes, Probe function can dynamically instrument the kernel	No
Version	Replaced oprofile in upstream linux kernel 3.0+	Not supported by linux kernel 3.0+

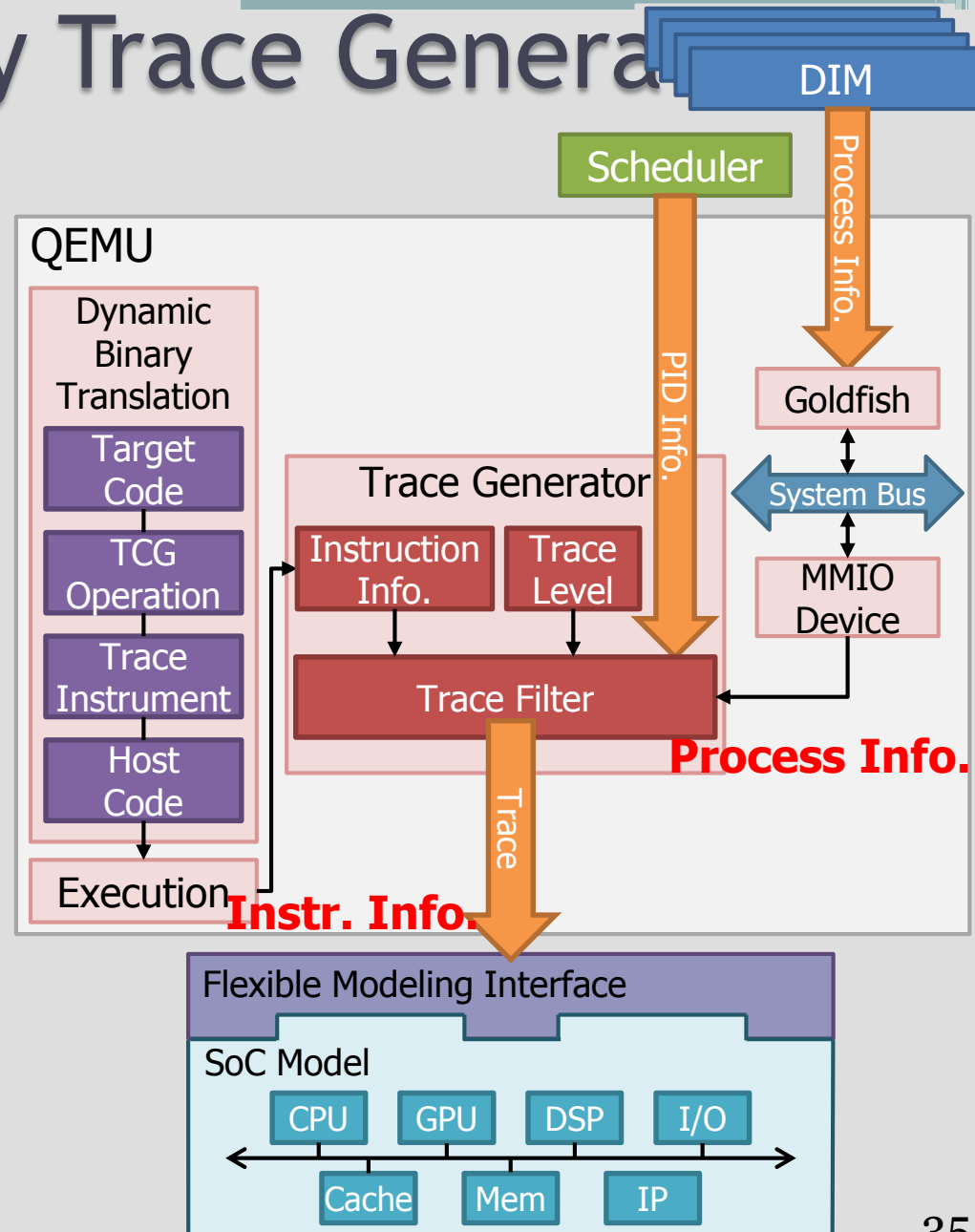
Profiling-guided SoC ESL exploration by Qemu simulation

- DIM module
 - Pass app info. through DVM
 - Memory tunnel send info. across barrier
- Multi-granularity Trace Generator
 - Trace instrument in DBT
 - Memory mapping I/O device
 - Support four levels of info.
- Flexible ESL Modeling Interface
 - Connect to SoC model for exploration and verification
 - Trace-driven simulation for SoC model



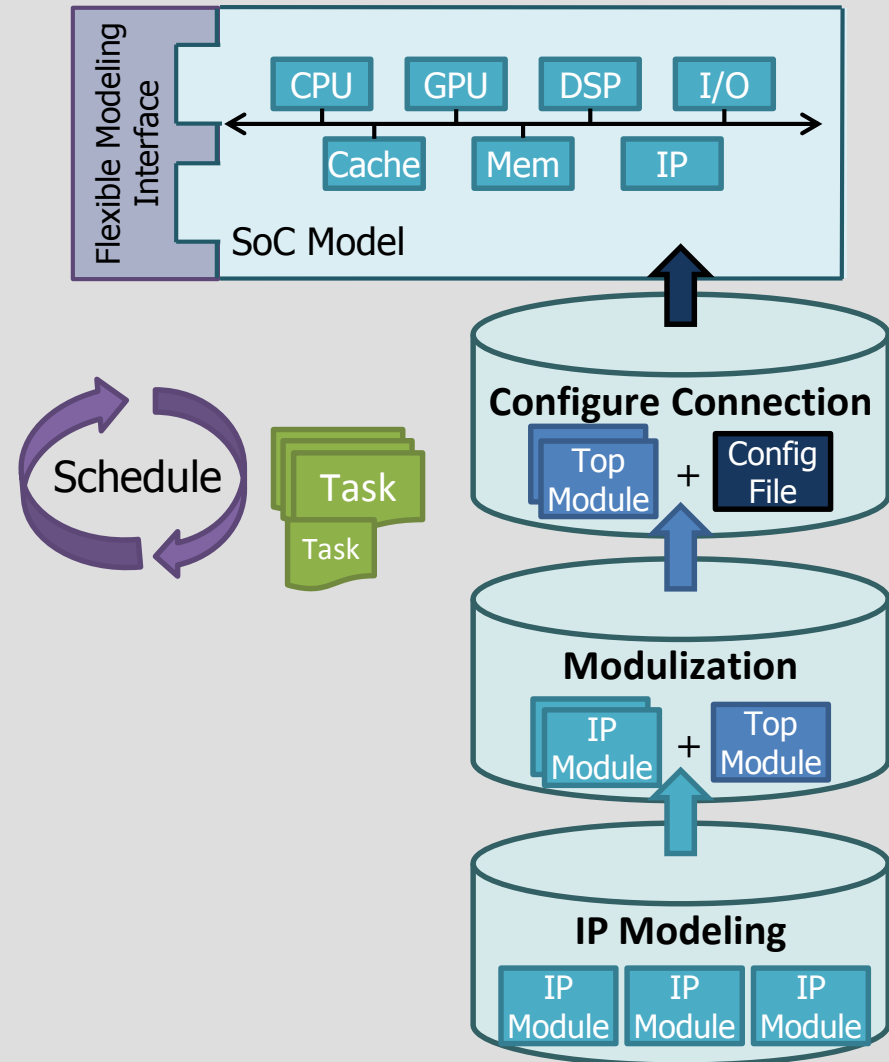
Multi-granularity Trace Generation

- Trace instrument in DBT
 - Profile process to instr. level precision
 - Filter useful instruction info. during host code execution
- Memory mapping I/O device
 - Pass process info. from DIM to trace generator
 - Control trace On/Off
- Support four levels of info.
 - Kernel level
 - User level
 - Process level
 - Method level



Flexible ESL Modeling Interface

- Flexible interface
 - Receive task from trace generator
 - Feed task to proper module in SoC model
- SoC model
 - Trace-driven simulation
 - Build each IP module according to specification
 - IP module is wrapped in top module to form a package
 - Configuration file declaring inter-connections forms target architecture



Testing Platform and Evaluation

Development Board		
Beagleboard	xM	
CPU	TI DM3730 ARM Cortex A8 1GHz	
CPU Arch.	CPU: ARM v7	
Cache	L1: 64KB (I\$+D\$)	L2: 256KB
Memory	NAND: N/A	

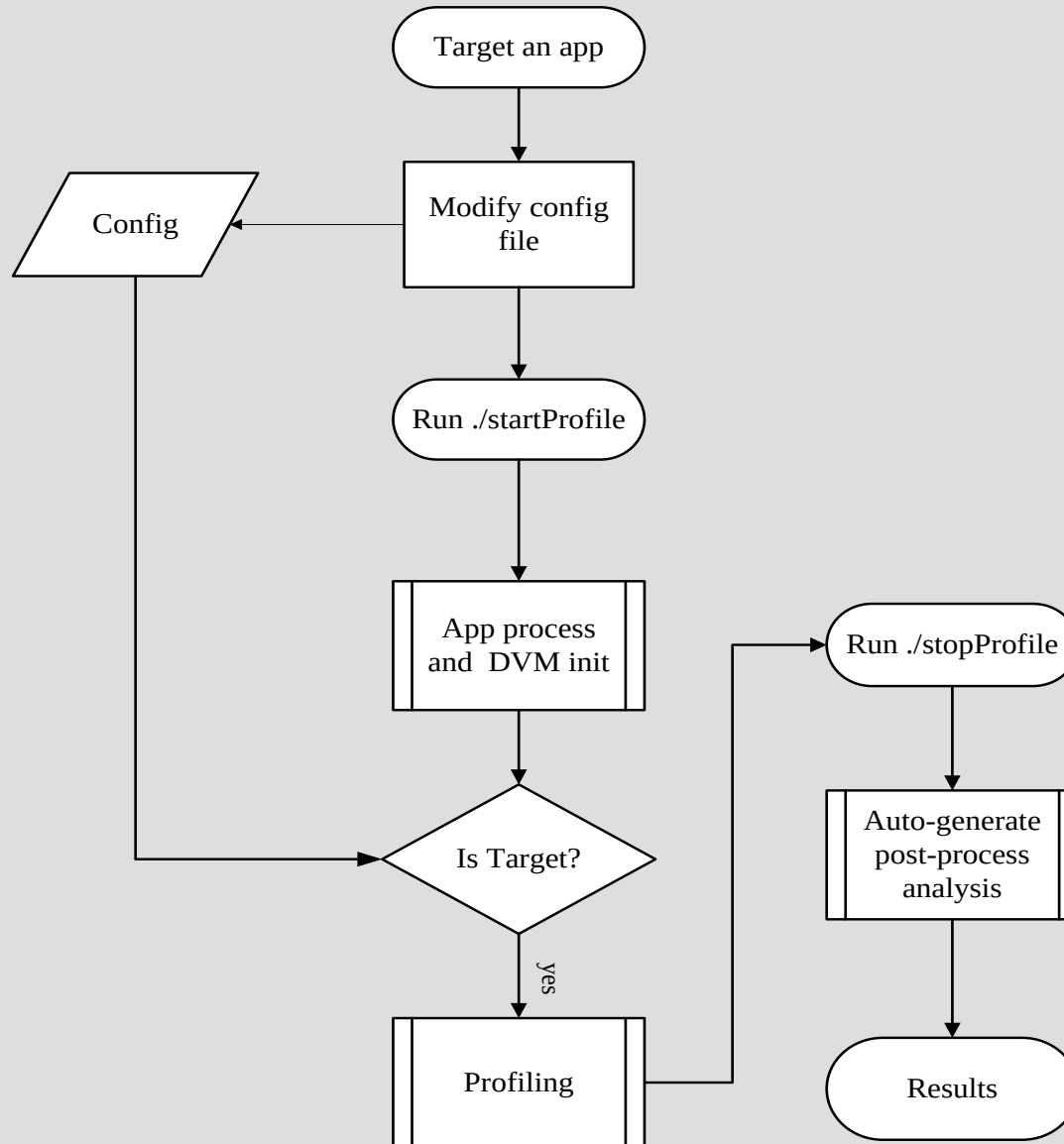


Software Environment

Experimental Environment		
Android	4.1.2 Gingerbread	
Kernel	2.6.37	
Compiler	gcc version 4.6	
Oprofile	0.9.6	
Benchmarks	Newton's Cradle	
	0xBench Math	Linpack
		Scimark2
	0xBench VM	



Profiling Process Flow



Configuration File/Profiling

```
ProcessName = [ "org.zerolab.benchmark", "com.geekyouup.android.newton",  
"org.teacake.monolith.apk" ]
```

```
MethodName = ["getNativePaint", "getBenchmark"]
```

Config file

- Utilizes liblcfg:
(lightweight configuration file library)
- Easy to configure
- Profiling process already automated with scripts

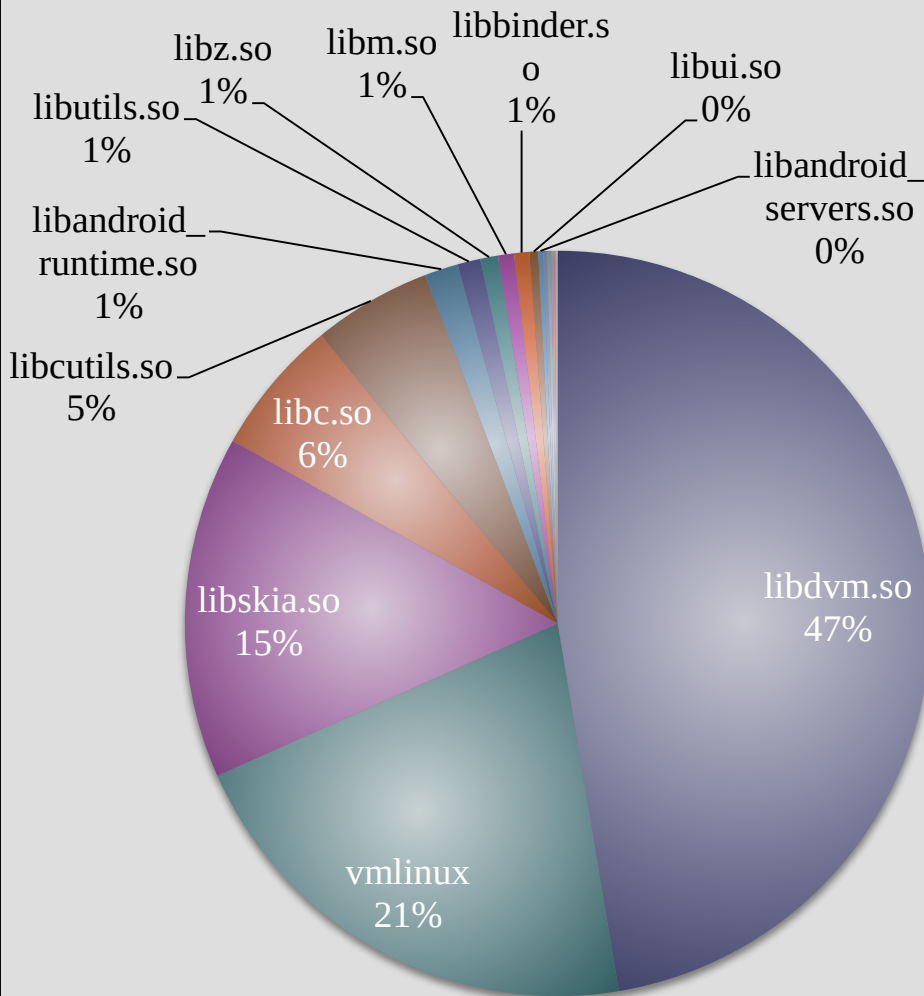
Vertical Profiling with Backtrace Sample Row File

1	4.5455	javasample.jo	javasample.jo	Lcom/android/internal/util/FastXmlSerializer;append:FastXmlSerializer.java:113%100
1	4.5455	javasample.jo	javasample.jo	Landroid/view/ViewRootImpl;draw:ViewRootImpl.java:2065
1	4.5455	javasample.jo	javasample.jo	Landroid/view/ViewRootImpl;collectViewAttributes:ViewRootImpl.java:0%100
1	4.5455	javasample.jo	javasample.jo	Landroid/view/ViewGroup;dispatchDraw:ViewGroup.java:2850%100~6
1	4.5455	javasample.jo	javasample.jo	Landroid/view/View;onCreateDrawableState:View.java:0%100
1	4.5455	javasample.jo	javasample.jo	Ljava/io/FileInputStream;read:FileInputStream.java:179%100
1	4.5455	javasample.jo	javasample.jo	Landroid/view/ViewTreeObserver;dispatchOnPreDraw:ViewTreeObserver.java:665%100
1	4.5455	javasample.jo	javasample.jo	Landroid/view/ViewRootImpl;performTraversals:ViewRootImpl.java:1578%100
1	4.5455	javasample.jo	javasample.jo	Landroid/view/Choreographer;doCallbacks:Choreographer.java:559%100
1	4.5455	javasample.jo	javasample.jo	Landroid/os/Message;markInUse:Message.java:424%100
1	4.5455	javasample.jo	javasample.jo	Landroid/os/Looper;loop:Looper.java:147%100~1
1	4.5455	javasample.jo	javasample.jo	Landroid/graphics/Canvas;drawRect:Canvas.java:920
1	4.5455	javasample.jo	javasample.jo	Landroid/graphics/Canvas;drawRect:Canvas.java:935%100
1	4.5455	javasample.jo	javasample.jo	Landroid/view/ViewGroup;dispatchDraw:ViewGroup.java:2850%100~10
1	4.5455	javasample.jo	javasample.jo	Ldalvik/system/BlockGuard\$1;onReadFromDisk:BlockGuard.java:120
2	9.0909	javasample.jo	javasample.jo	Landroid/view/View;draw:View.java:13193%100
2	9.0909	javasample.jo	javasample.jo	Landroid/view/ViewTreeObserver;dispatchOnDraw:ViewTreeObserver.java:681%100
3	13.6364	javasample.jo	javasample.jo	Landroid/view/ViewRootImpl;performTraversals:ViewRootImpl.java:1461%100
0	0	libdvm.so	javasample.jo	dvmCheckCallJNIMethod(unsigned int const*, JValue*, Method const*, Thread*)
0	0	libdvm.so	javasample.jo	dvmCheckCallJNIMethod(unsigned int const*, JValue*, Method const*, Thread*) [self]
1	33.3333	javasample.jo	javasample.jo	Landroid/os/Message;recycle:Message.java:249%100
1	33.3333	javasample.jo	javasample.jo	Landroid/view/Choreographer;postCallbackDelayed:Choreographer.java:285%100
1	33.3333	javasample.jo	javasample.jo	Landroid/view/ViewRootImpl;invalidateChildInParent:ViewRootImpl.java:906%100
0	0	libdvm.so	javasample.jo	dvmFindInterfaceMethodInCache
0	0	libdvm.so	javasample.jo	dvmFindInterfaceMethodInCache [self]

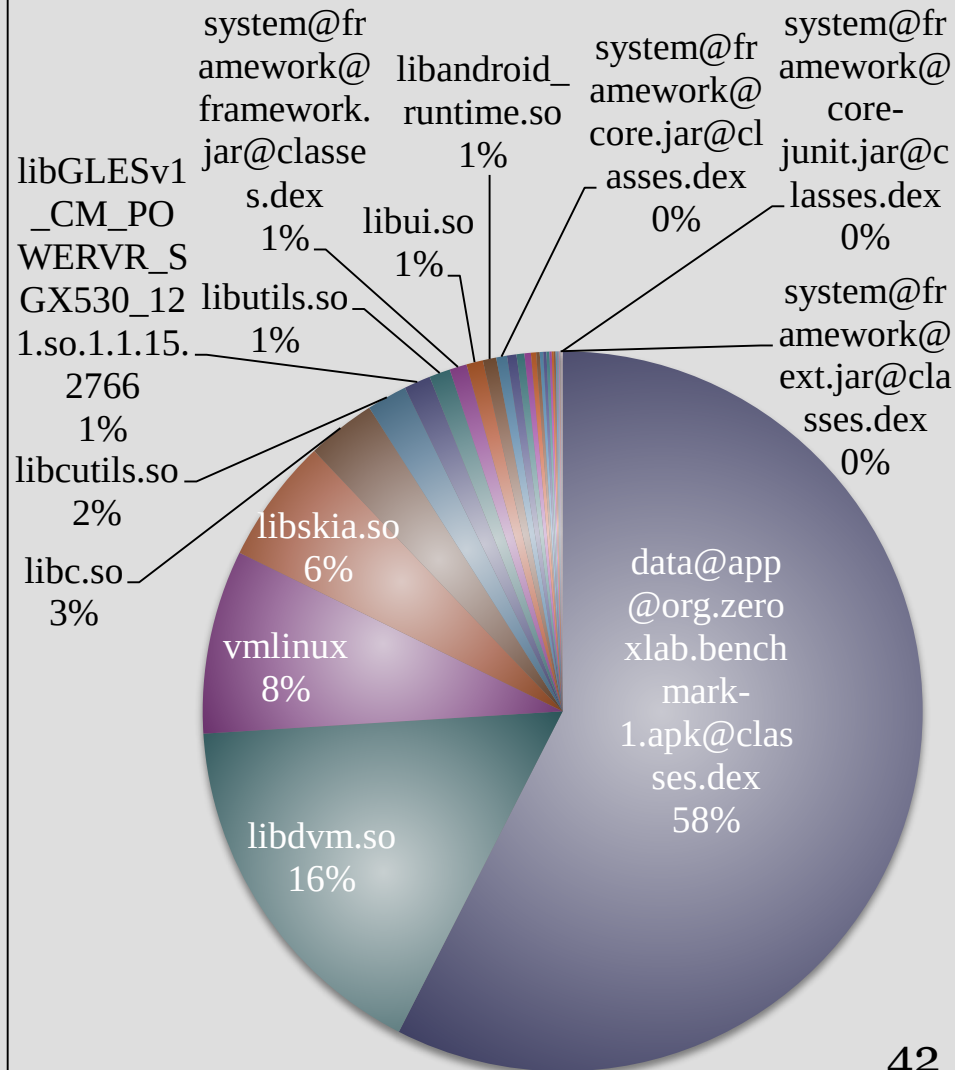
Sample result for Android I/O Benchmark

Oprofile, VARI profile Comparison

0xBench Math (without VARI)

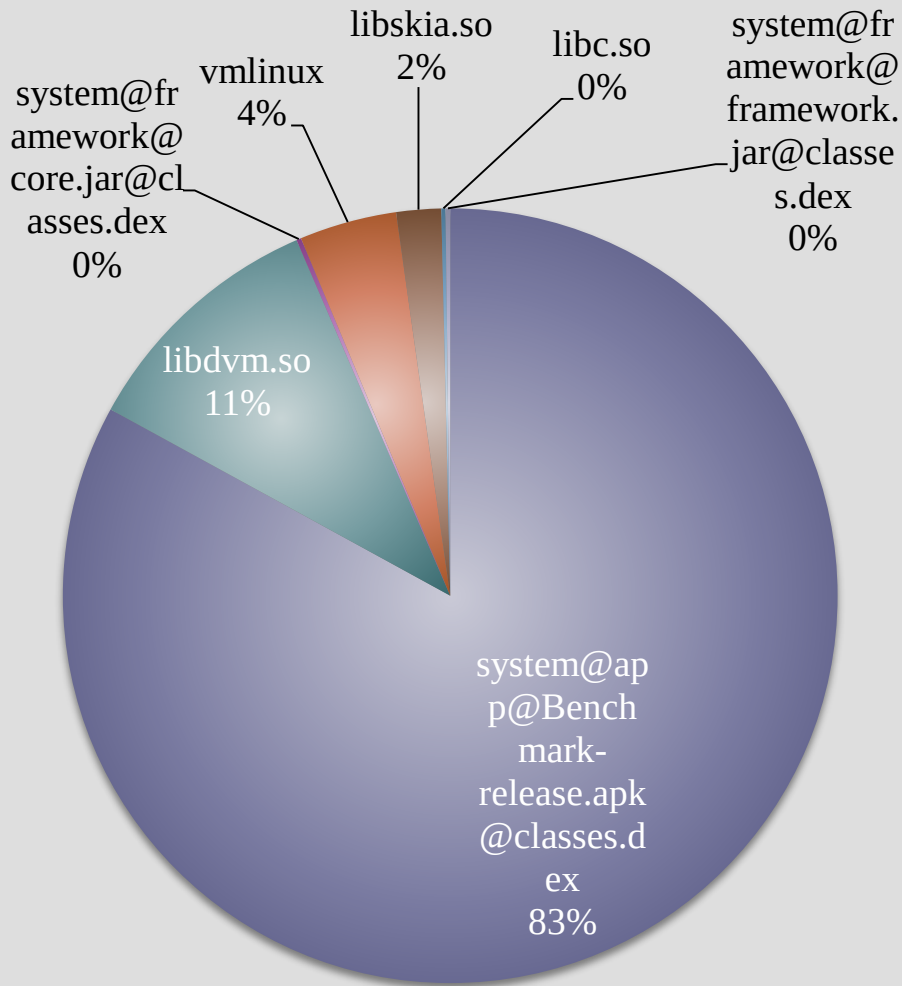


0xBench Math (VARI)

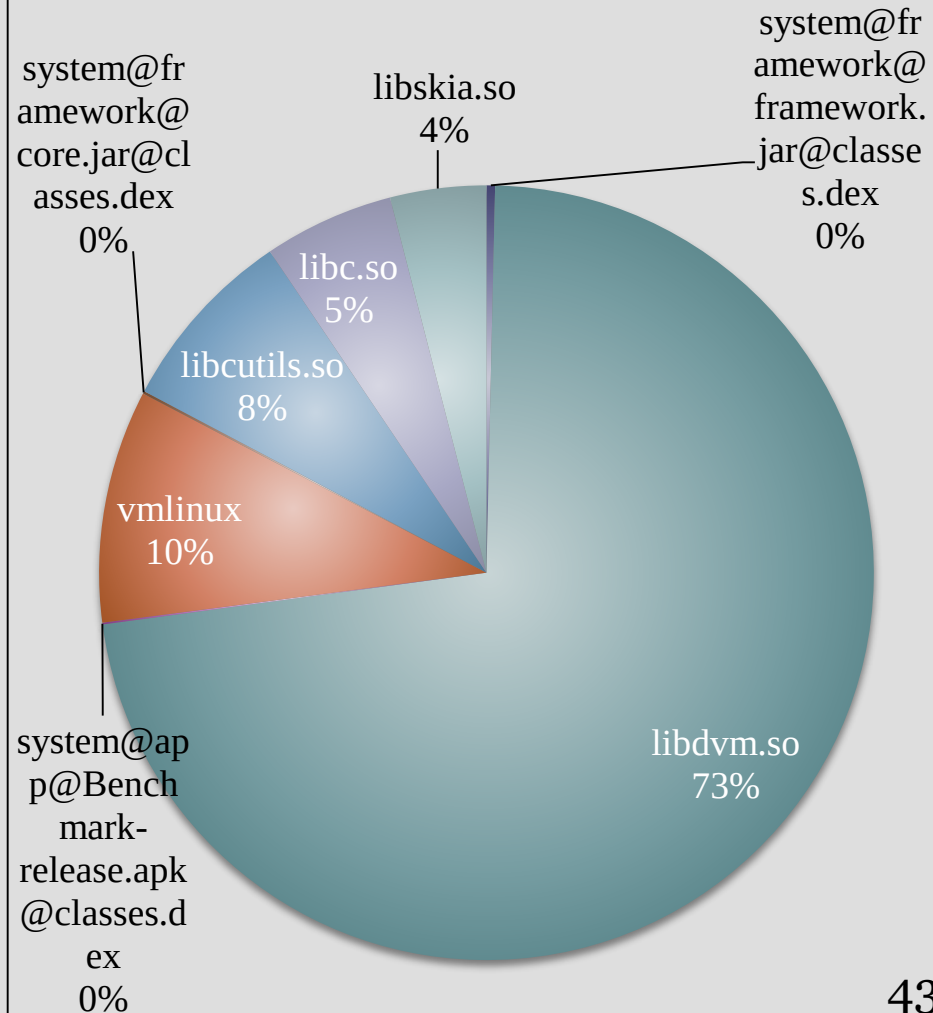


VARI Profiling Result Analysis

0xBench Math (VARI)



0xBench VM (VARI)



Proc Map Example

DIM instrumented results of Newton's cradle

File	Function	rPC	VMA	Method Name	Class Description	Source File Name	Line#
Interp.c	dvmInterpFindInterfaceMethod	6	0x42729240	run	Ljava/lang/Runnable;	ViewRoot.java	2044
Resolve.c	dvmResolveMethod	636	0x43966df0	atan	Ljava/lang/Math;	BallsView.java	616
Jni.c	dvmCallJNIMethod_general	725	0x254608	native_drawBitmap	Landroid/graphics/Canvas;	Canvas.java	-2

proc/pid/maps

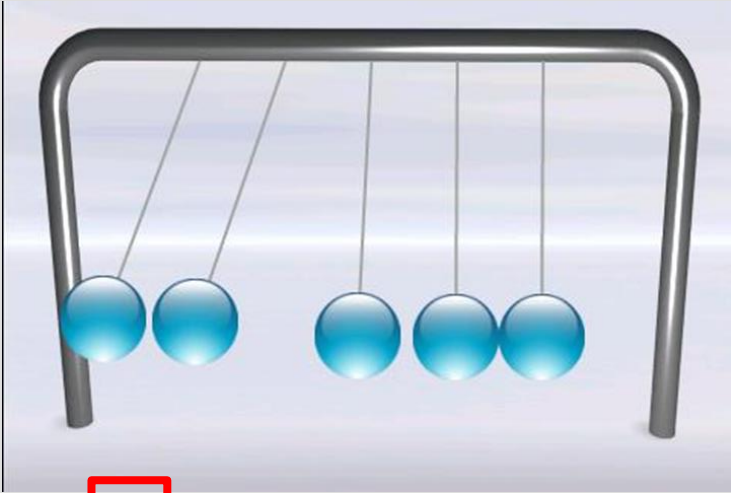
Start Addr.	End Addr.	Mod	Node	Mapped File Name
0000a000	002bd000	rwxp	0	[heap]
4245b000	42b01000	r-xp	293	/data/dalvik-cache/system@framework@framework.jar@classes.dex
43964000	4396b000	r-xp	723	/data/dalvik-cache/data@app@com.geekyouup.android.newton-1.apk@classes.dex

Source Line Correlation

```
603 for(int p=0;p<numPointers;p++)
604 {
605     if(mObjectTouched && mObjectsTouchedIds[p]!=-1)
606     {
607         mBallVelocity[mObjectsTouchedIds[p]]=0;
608
609         //translate the roational origin
610         float transX = reflectGetXForPointer(event, p)-mCenterOfRotationX[mObjectsTouchedIds[p]];
611         float transY = reflectGetYForPointer(event, p)-mCenterOfRotationY;
612
613         if(!isOrientNormal) {transX=-transX; transY=-transY;}
614
615         //figure out the angle the ball is now at
616         if(transY>0) mBallAngle[mObjectsTouchedIds[p]] = ((float) (-Math.atan(transX/transY))+PI_F);
617         else mBallAngle[mObjectsTouchedIds[p]] = (float) -Math.atan((transX/transY));
618     }
619 }
```

BallsView.java

Opreport Result for Newton



The 1st is line 414

813:401017f:10	0.1949	/data/local/oprofile/javasample.jo	Lcom/geekyouup/android/newton/BallsView\$BallsThread;hitTestBallOverRange:BallsView.java:414%100-3
868:4010146:7	0.1365	/data/local/oprofile/javasample.jo	Lcom/geekyouup/android/newton/BallsView\$BallsThread;hitTestBallOverRange:BallsView.java:393
921:4010144:6	0.1170	/data/local/oprofile/javasample.jo	Lcom/geekyouup/android/newton/BallsView\$BallsThread;hitTestBallOverRange:BallsView.java:391%100
995:4010175:5	0.0975	/data/local/oprofile/javasample.jo	Lcom/geekyouup/android/newton/BallsView\$BallsThread;hitTestBallOverRange:BallsView.java:414-1
1055:401014c:4	0.0780	/data/local/oprofile/javasample.jo	Lcom/geekyouup/android/newton/BallsView\$BallsThread;hitTestBallOverRange:BallsView.java:392
1057:4010160:4	0.0780	/data/local/oprofile/javasample.jo	Lcom/geekyouup/android/newton/BallsView\$BallsThread;hitTestBallOverRange:BallsView.java:414%100-2
1148:4010136:3	0.0585	/data/local/oprofile/javasample.jo	Lcom/geekyouup/android/newton/BallsView\$BallsThread;hitTestBallOverRange:BallsView.java:377%100
1150:4010178:3	0.0585	/data/local/oprofile/javasample.jo	Lcom/geekyouup/android/newton/BallsView\$BallsThread;hitTestBallOverRange:BallsView.java:414%100
1305:4010142:2	0.0390	/data/local/oprofile/javasample.jo	Lcom/geekyouup/android/newton/BallsView\$BallsThread;hitTestBallOverRange:BallsView.java:389%0%100
1307:4010168:2	0.0390	/data/local/oprofile/javasample.jo	Lcom/geekyouup/android/newton/BallsView\$BallsThread;hitTestBallOverRange:BallsView.java:414-5
1678:4010146:1	0.0195	/data/local/oprofile/javasample.jo	Lcom/geekyouup/android/newton/BallsView\$BallsThread;hitTestBallOverRange:BallsView.java:389
1680:4010162:1	0.0195	/data/local/oprofile/javasample.jo	Lcom/geekyouup/android/newton/BallsView\$BallsThread;hitTestBallOverRange:BallsView.java:414-7

Symbol hit times

Symbol source line

```

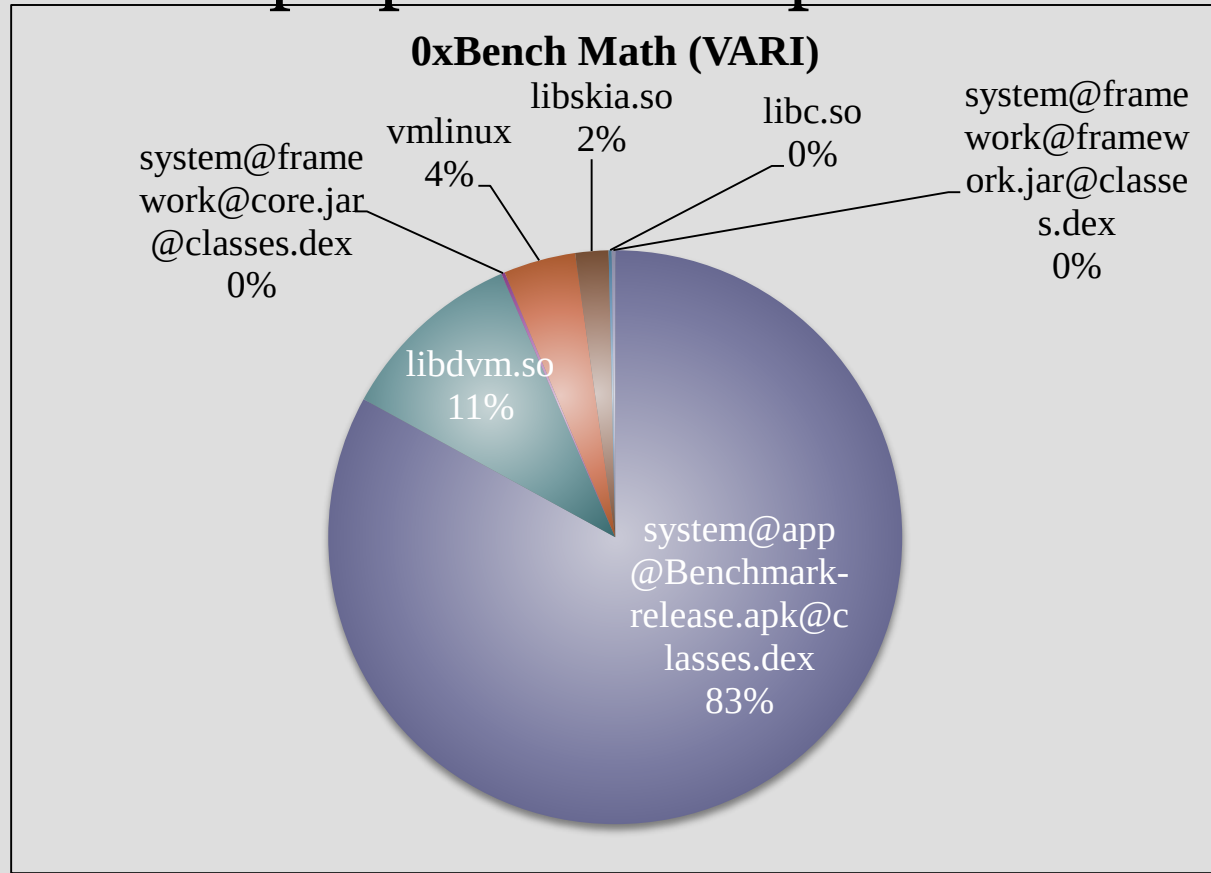
413: if(!mObjectTouched && mVelocityDiff > 0 && testBall>1) hitTestBallOverRange(testBall, testBall-2, testBall-1, true);
414:     else if(!mObjectTouched && mVelocityDiff < 0 && (testBall<mNumberOfBalls-1)) hitTestBallOverRange(testBall, testBall, testBall+1, false);
  
```

It's a recursive function!!

Automated Graphical Profiling Analysis

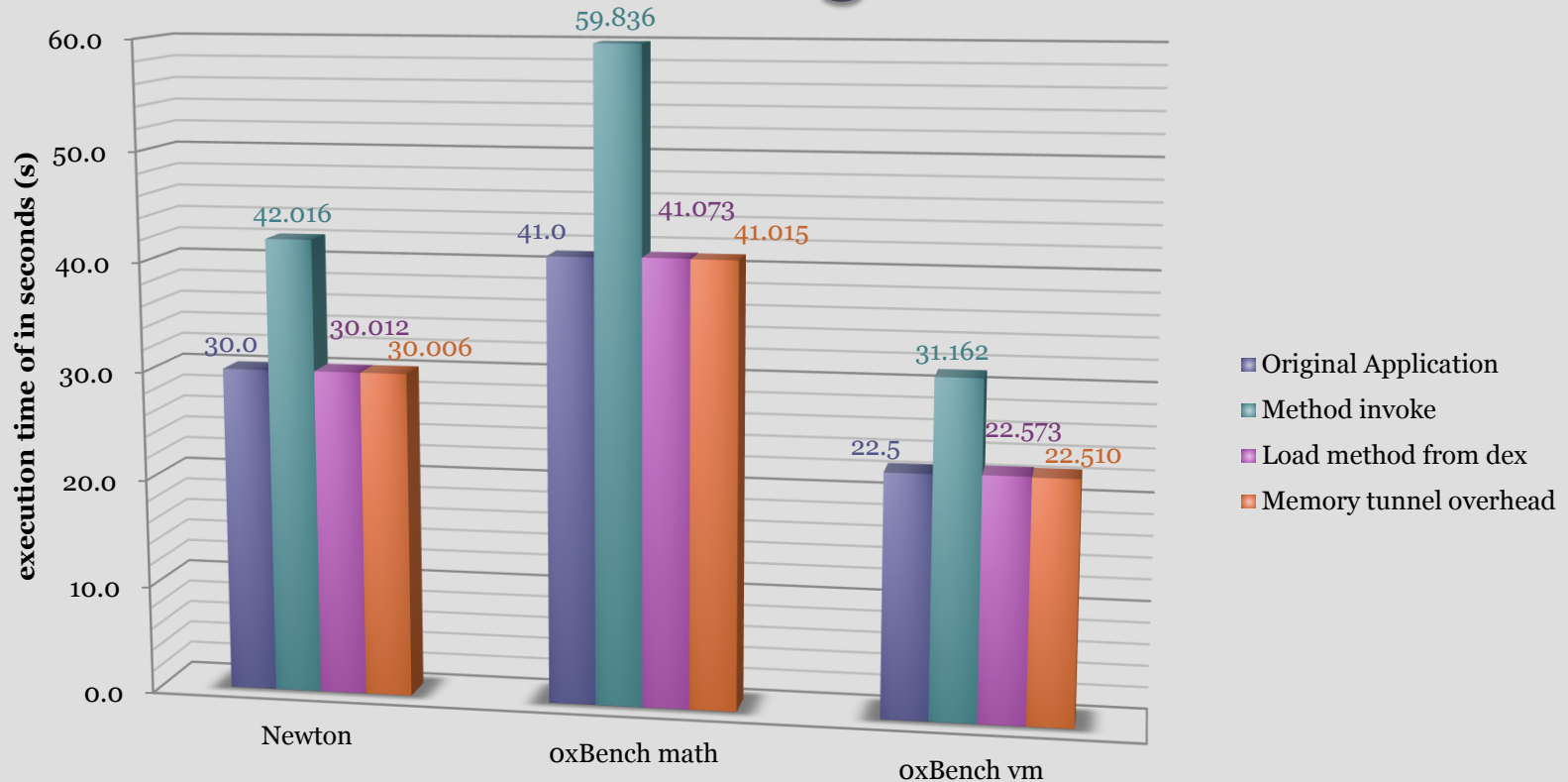
- Automated tool from opreport text outputs:

- Pie Chart
- Call graph



app_process system@app@Benchmark-release.apk@classes.dex getTitle: CaseTeapot.java:48 16.04% (16.04%)	app_process vmlinux /data/local/vmlinux 2.67% (2.67%)	app_process system@app@Benchmark-release.apk@classes.dex getDescription: CaseNehLesson08.java:58 18.49% (18.49%)	vmlinux vmlinux /data/local/vmlinux 14.78% (14.78%)	app_process libandroid_runtime.so android:register_android_content_res_Configuration 1.51% (0.00%)	app_process system@app@Benchmark-release.apk@classes.dex CaseGLCube.java:37 18.32% (18.32%)	app_process system@app@Benchmark-release.apk@classes.dex getBenchmark: CaseTeapot.java:57 15.33% (15.33%)
--	--	---	--	---	--	--

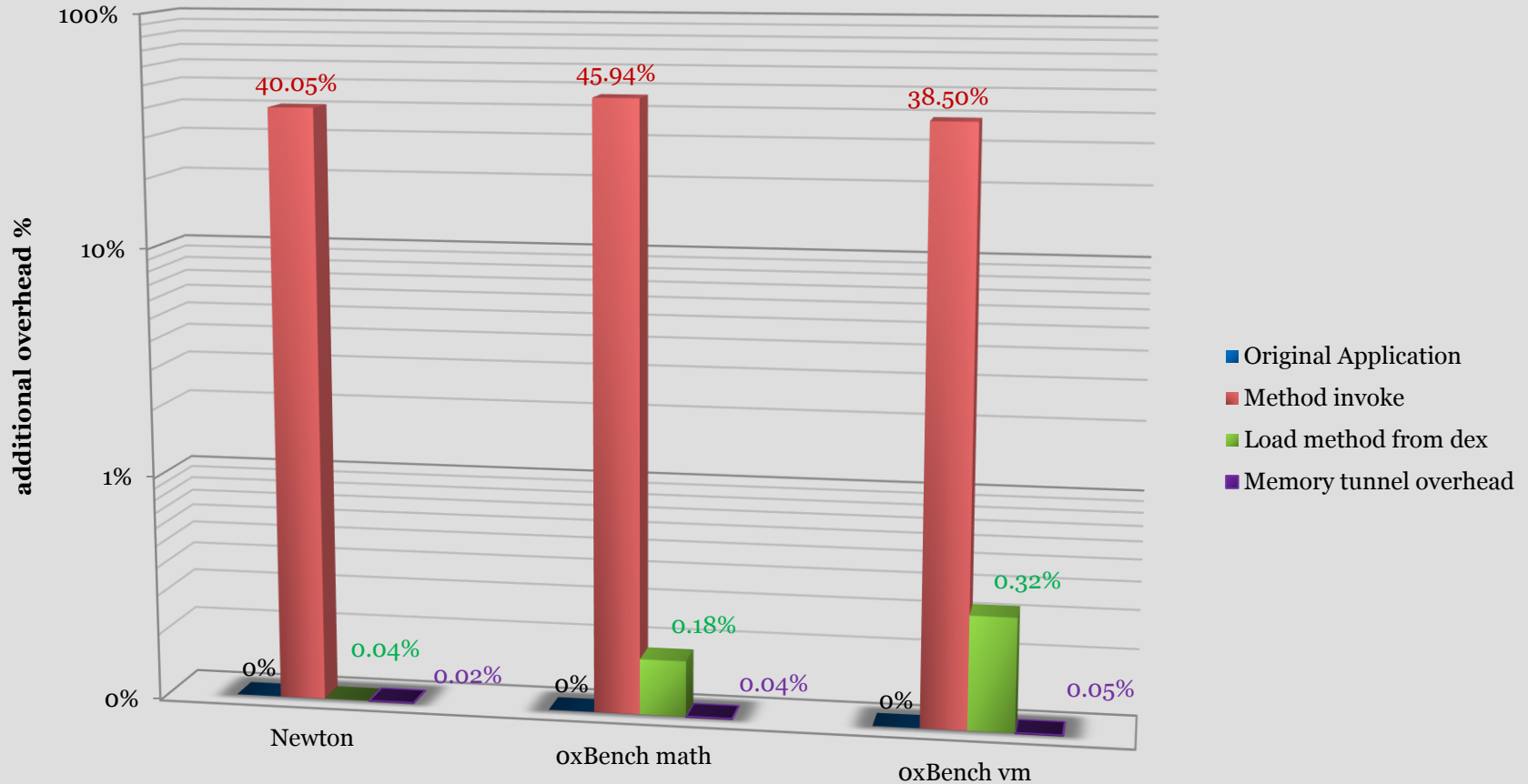
Overhead of different DIM instrument methodologies



Overhead of different DIM instrument methodologies over 3 benchmarks

	Original Application	Method invoke	Load method from dex	Memory tunnel overhead
Newton	30.0	42.016	30.012	30.006
oxBench math	41.0	59.836	41.073	41.015
oxBench vm	22.5	31.162	22.573	22.510

Overhead Comparison

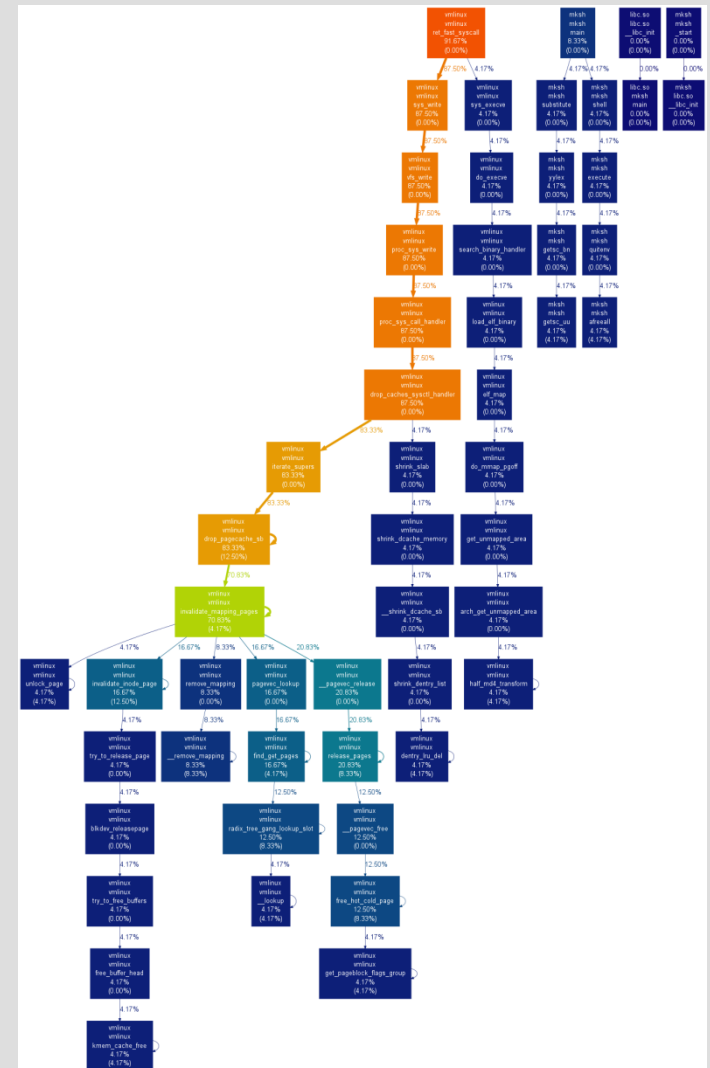


Overhead of different DIM instrument methodologies over 3 benchmarks

Overhead	Original Application	Method invoke	Load method from dex	Memory tunnel overhead
Newton	0.00%	40.05%	0.04%	0.02%
oxBench math	0.00%	45.94%	0.18%	0.04%
oxBench vm	0.00%	38.50%	0.32%	0.05%

A Profiling Example Based on Backtrace

- Identify a hot zone from full system profiling
- Zoom in to process level profiling



Perf tools example with VARI

```
# Events: 122 cycles
#
# Overhead  Command      Shared Object      Symbol
# .....  ....
#
41.27% Binder_7 [unknown]          [.] 0x58622b1c
15.90% Binder_7 fakeElf.jo         [k] ktime_get_ts
10.65% Binder_7 fakeElf.jo         [k] _raw_spin_unlock_irqrestore
8.16% Binder_7 fakeElf.jo         [k] finish_task_switch
7.95% Binder_7 fakeElf.jo         [k] sys_futex
7.95% Binder_7 fakeElf.jo         [k] plist_check_head
1.76% Binder_7 fakeElf.jo         [k] __do_softirq
1.43% Binder_7 fakeElf.jo         [k] omap_bus_sync
1.41% Binder_7 fakeElf.jo         [k] update_shares
1.41% Binder_7 fakeElf.jo         [k] SGXDoKickBW
1.25% Binder_7 fakeElf.jo         [k] __mutex_unlock_slowpath
0.43% Binder_7 fakeElf.jo         [k] __copy_from_user
0.26% Binder_7 fakeElf.jo         [k] schedule
0.18% Binder_7 fakeElf.jo         [k] do_mmap_pgoff

#
# (For a higher level overview, try: perf report --sort comm,dso)
#
```



0x58622b1c|guardedRun:GLSurfaceView.java:1296
0x58622b1c|guardedRun:GLSurfaceView.java:1296
0x58622b1c|guardedRun:GLSurfaceView.java:1296
0x58622b1c|guardedRun:GLSurfaceView.java:1296
0x58622b1c|guardedRun:GLSurfaceView.java:1296
0x58622b1c|guardedRun:GLSurfaceView.java:1296



App Performance Eval

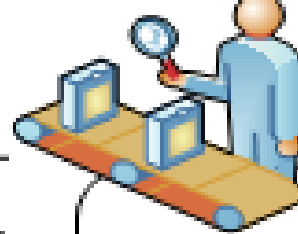
Evaluate performance of critic



System Software Verifi

When Android system software is under performing, developers can propose vertical profiler to verify cause.

- Kernel regression
- Driver improvement
- Improve hardware (cache, DSP, GPU, etc.) utilization
- Verify DVM efficiency



Platform Provider Support

FAE can help customer application developers identify whether a bottleneck is in customer code or system software.

- Quickly determine responsibility of poor performance
- Assist customer developers



• Evaluate hardware architectures
• Evaluate hardware components
• Verify a hardware design

Conclusion

- VARI profiler provides vertical profiling capability on Android mobile systems
- Bridge the profiling gap between the Dalvik virtual machine and Linux user and kernel space
- Identify Java/system level bottlenecks
- Trace back to the point of origin in the Linux system library
 - Hash remapping address algorithm
- Introduce little overhead without modification
- Build Qemu-driven SoC simulation to enable profiling-guided SoC ESL exploration

Thank You

